

Amazon bestseller in multiple categories.

ENCOMENDE ONLINE

Also available in Chinese, Japanese, German, Portuguese, Korean, French, Arabic, and Russian... more coming soon.

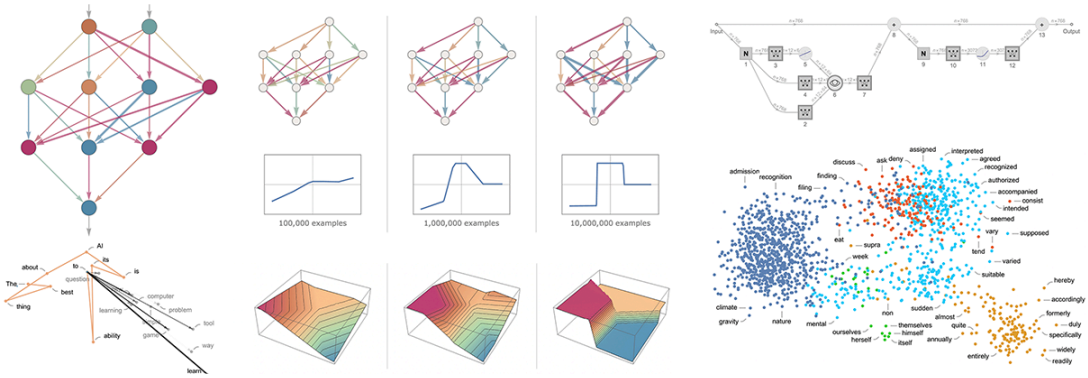
Veja também:

"A tecnologia LLM chega à Wolfram Language" »

Uma discussão sobre a história das redes neurais »

O que o ChatGPT faz... e por que funciona?

14 de fevereiro de 2023



É só adicionar uma palavra de cada vez.

O fato de o ChatGPT conseguir gerar automaticamente algo que, mesmo superficialmente, se assemelha a um texto escrito por humanos é notável e inesperado. Mas como ele faz isso? E por que funciona? Meu objetivo aqui é apresentar um panorama geral do que acontece internamente no ChatGPT e, em seguida, explorar por que ele consegue produzir com tanta eficiência o que podemos considerar um texto significativo. Devo dizer desde já que vou me concentrar no panorama geral do processo e, embora mencione alguns detalhes técnicos, não me aprofundarei neles. (E a essência do que direi se aplica igualmente a outros "grandes modelos de linguagem" [LLMs] atuais, assim como ao ChatGPT.)

fundamentalmente é produzir uma "continuação razoável" de qualquer texto que tenha recebido até o momento, onde por "razoável" queremos dizer "o que se poderia esperar que alguém escrevesse depois de ver o que as pessoas escreveram em bilhões de páginas da web, etc."

Digamos que temos o texto " *A melhor coisa sobre IA é sua capacidade de...* ". Imagine analisar bilhões de páginas de texto escrito por humanos (digamos, na web e em livros digitalizados) e encontrar todas as ocorrências desse texto — e então verificar qual palavra vem a seguir e com que frequência. O ChatGPT faz algo parecido, só que (como explicarei) ele não analisa o texto literalmente; ele busca por coisas que, de certa forma, "correspondem em significado". Mas o resultado final é uma lista classificada de palavras que podem vir a seguir, juntamente com as "probabilidades":

The best thing about AI is its ability to

learn	4.5%
predict	3.5%
make	3.2%
understand	3.1%
do	2.9%

E o mais notável é que, quando o ChatGPT faz algo como escrever uma redação, o que ele essencialmente faz é perguntar repetidamente: "Dado o texto até agora, qual deve ser a próxima palavra?" — e a cada vez, adiciona uma palavra. (Mais precisamente, como explicarei, ele adiciona um "token", que pode ser apenas parte de uma palavra, e é por isso que às vezes ele pode "inventar novas palavras".)

Mas, tudo bem, a cada etapa o programa recebe uma lista de palavras com probabilidades. Mas qual delas ele deve escolher para adicionar à redação (ou o que for) que está escrevendo? Alguém poderia pensar que deveria ser a palavra com a "maior probabilidade" (ou seja, aquela à qual foi atribuída a maior probabilidade). Mas é aqui que a mágica começa a aparecer. Porque, por algum motivo — que talvez um dia venhamos a entender cientificamente —, se sempre escolhermos a palavra com a maior probabilidade, geralmente obteremos uma redação muito "sem graça", que nunca parece "mostrar criatividade" (e às vezes até repete palavra por palavra). Mas se, às vezes (aleatoriamente), escolhermos palavras com probabilidades mais baixas, obtemos uma redação "mais interessante".

O fato de haver aleatoriedade aqui significa que, se usarmos o mesmo tema várias vezes, provavelmente obteremos redações diferentes a cada vez. E, seguindo a ideia de vodu, existe um parâmetro específico chamado "temperatura" que determina a frequência com que palavras de menor relevância serão usadas, e, para a geração de redações, uma "temperatura" de 0,8 parece ser a melhor opção. (Vale ressaltar que não há nenhuma "teoria" sendo usada aqui; trata-se apenas do que se mostrou eficaz na

exponenciais, familiares da física estatística, estão sendo usadas, mas não há nenhuma conexão "física" — pelo menos até onde sabemos.)

Antes de prosseguirmos, devo explicar que, para fins de exposição, geralmente não utilizarei o sistema completo do ChatGPT ; em vez disso, trabalharei com um sistema GPT-2 mais simples , que tem a vantagem de ser pequeno o suficiente para ser executado em um computador desktop padrão. Portanto, para praticamente tudo o que eu mostrar, incluirei código Wolfram Language explícito que você poderá executar imediatamente em seu computador. (Clique em qualquer imagem aqui para copiar o código correspondente.)

Por exemplo, veja como obter a tabela de probabilidades acima. Primeiro, precisamos recuperar a rede neural subjacente do "modelo de linguagem" :

```
In[ ]:= model =
          NetModel[{"GPT2 Transformer Trained on WebText Data",
            "Task" → "LanguageModeling"}]
```

```
Out[ ]= NetChain[ Input port:  string
                  Output port: class ]
```

Mais tarde, analisaremos o funcionamento interno dessa rede neural. Mas, por enquanto, podemos simplesmente aplicar esse "modelo de rede" como uma caixa preta ao nosso texto atual e solicitar as 5 palavras mais prováveis que o modelo indica que devem vir a seguir:

```
In[ ]:= model["The best thing about AI is its ability to", {"TopProbabilities", 5}]
```

```
Out[ ]= {do → 0.0288508, understand → 0.0307805,
           make → 0.0319072, predict → 0.0349748, learn → 0.0445305}
```

Isso pega esse resultado e o transforma em um " conjunto de dados " formatado explicitamente :

```
In[ ]:= Dataset[ReverseSort[Association[%]],
          ItemDisplayFunction → (PercentForm[#, 2] &)]
```

Out[ ]=

learn	4.5%
predict	3.5%
make	3.2%
understand	3.1%
do	2.9%

Eis o que acontece se alguém "aplicar o modelo" repetidamente — adicionando a cada passo a palavra que tem a maior probabilidade (especificada neste código como a "decisão" do modelo):

```
"The best thing about AI is its ability to", 7]
```

```
Out[ ]= {The best thing about AI is its ability to,
         The best thing about AI is its ability to learn,
         The best thing about AI is its ability to learn from,
         The best thing about AI is its ability to learn from experience,
         The best thing about AI is its ability to learn from experience.,
         The best thing about AI is its ability to learn from experience. It,
         The best thing about AI is its ability to learn from experience. It's,
         The best thing about AI is its ability to learn from experience. It's not}
```

O que acontece se isso se prolongar por mais tempo? Nesse caso (temperatura zero), o resultado logo se torna bastante confuso e repetitivo:

```
The best thing about AI is its ability to learn from experience. It's not just a matter
of learning from experience, it's learning from the world around you. The AI is a very
good example of this. It's a very good example of how to use AI to improve your life.
It's a very good example of how to use AI to improve your life. The AI is a very good
example of how to use AI to improve your life. It's a very good example of how to use AI to
```

Mas e se, em vez de escolher sempre a palavra "principal", alguém escolhesse aleatoriamente palavras "não principais" (com a "aleatoriedade" correspondendo à "temperatura" de 0,8)? Novamente, é possível construir o texto:

```
{The best thing about AI is its ability to,
 The best thing about AI is its ability to create,
 The best thing about AI is its ability to create worlds,
 The best thing about AI is its ability to create worlds that,
 The best thing about AI is its ability to create worlds that are,
 The best thing about AI is its ability to create worlds that are both,
 The best thing about AI is its ability to create worlds that are both exciting,
 The best thing about AI is its ability to create worlds that are both exciting,}
```

E cada vez que isso for feito, diferentes escolhas aleatórias serão feitas, e o texto será diferente — como nestes 5 exemplos:

```
The best thing about AI is its ability to learn. I've always liked the
```

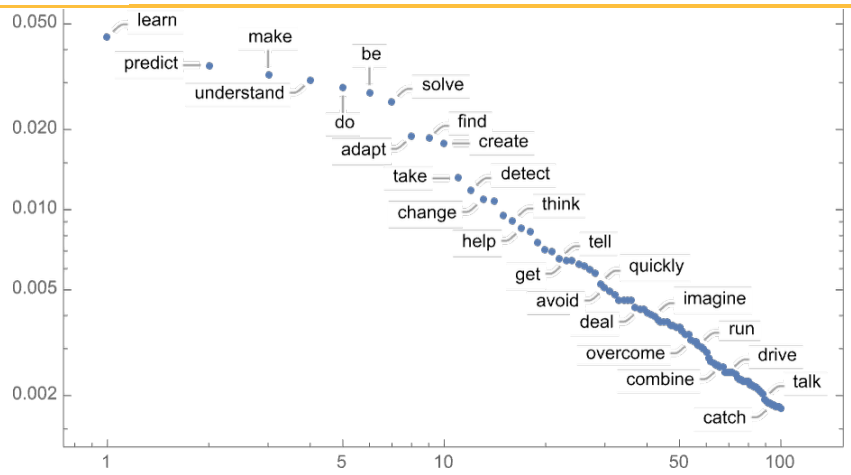
```
The best thing about AI is its ability to really come into your world and just
```

```
The best thing about AI is its ability to examine human behavior and the way it
```

```
The best thing about AI is its ability to do a great job of teaching us
```

```
The best thing about AI is its ability to create real tasks, but you can
```

Vale ressaltar que, mesmo no primeiro passo, há muitas “próximas palavras” possíveis para escolher (na temperatura 0,8), embora suas probabilidades diminuam rapidamente (e, sim, a linha reta neste gráfico log-log corresponde a um decaimento em “lei de potência” n^{-1}, que é muito característico das estatísticas gerais da linguagem):



Então, o que acontece se isso continuar por mais tempo? Aqui está um exemplo aleatório. É melhor do que o caso da palavra no topo (temperatura zero), mas ainda assim, no mínimo, um pouco estranho:

> The best thing about AI is its ability to see through, and make sense of, the world around us, rather than panicking and ignoring. This is known as AI "doing its job" or AI "run-of-the-mill." Indeed, taking an infinite number of steps, developing a machine that can be integrated with other systems, or controlling one system that's truly a machine, is one of the most fundamental processes of AI. Aside from the human-machine interaction, AI was also a big part of creativity

Isso foi feito com o modelo GPT-2 mais simples (de 2019). Com os modelos GPT-3 mais recentes e maiores, os resultados são melhores. Aqui está o texto da palavra principal (temperatura zero) produzido com o mesmo "estímulo", mas com o modelo GPT-3 maior:

> The best thing about AI is its ability to automate processes and make decisions quickly and accurately. AI can be used to automate mundane tasks, such as data entry, and can also be used to make complex decisions, such as predicting customer behavior or analyzing large datasets. AI can also be used to improve customer service, as it can quickly and accurately respond to customer inquiries. AI can also be used to improve the accuracy of medical diagnoses and to automate the process of drug discovery.

E aqui está um exemplo aleatório à "temperatura 0,8":

> The best thing about AI is its ability to learn and develop over time, allowing it to continually improve its performance and be more efficient at tasks. AI can also be used to automate mundane tasks, allowing humans to focus on more important tasks. AI can also be used to make decisions and provide insights that would otherwise be impossible for humans to figure out.

De onde vêm as probabilidades?

Certo, então o ChatGPT sempre escolhe a próxima palavra com base em probabilidades. Mas de onde vêm essas probabilidades? Vamos começar com um problema mais

palavra) por vez. Como podemos calcular qual deve ser a probabilidade para cada letra?

Uma coisa muito simples que poderíamos fazer é pegar uma amostra de texto em inglês e calcular a frequência com que diferentes letras ocorrem. Por exemplo, isso conta as letras no artigo da Wikipédia sobre "gatos":

```
In[ ]:= LetterCounts[WikipediaData["cats"]]

Out[ ]= ⟨| e → 4279, a → 3442, t → 3397, i → 2739, s → 2615, n → 2464, o → 2426,
  r → 2147, h → 1613, l → 1552, c → 1405, d → 1331, m → 989, u → 916,
  f → 760, g → 745, p → 651, y → 591, b → 511, w → 509, v → 395, k → 212,
  T → 114, x → 85, A → 81, C → 81, I → 68, S → 55, F → 42, z → 38, E → 36
```

E isto tem o mesmo efeito em relação aos "cães":

```
In[ ]:= LetterCounts[WikipediaData["dogs"]]

Out[ ]= ⟨| e → 3911, a → 2741, o → 2608, i → 2562, t → 2528, s → 2406,
  n → 2340, r → 1866, d → 1584, h → 1463, l → 1355, c → 1083, g → 929,
  m → 859, u → 782, f → 662, p → 636, y → 500, b → 462, w → 409,
  v → 406, k → 151, T → 90, C → 85, I → 80, A → 74, x → 71, S → 65,
```

Os resultados são semelhantes, mas não idênticos (o “o” é sem dúvida mais comum no artigo “dogs” porque, afinal, ocorre na própria palavra “dog”). Ainda assim, se analisarmos uma amostra suficientemente grande de textos em inglês, podemos esperar obter, eventualmente, resultados pelo menos razoavelmente consistentes:

```
In[ ]:= English LANGUAGE [ character frequencies ]

Out[ ]= {e → 12.7%, t → 9.06%, a → 8.17%, o → 7.51%, i → 6.97%, n → 6.75%,
  s → 6.33%, h → 6.09%, r → 5.99%, d → 4.25%, l → 4.03%, c → 2.78%, u → 2.76%,
  m → 2.41%, w → 2.36%, f → 2.23%, g → 2.02%, y → 1.97%, p → 1.93%, b → 1.49%,
  v → 0.978%, k → 0.772%, j → 0.153%, x → 0.150%, q → 0.0950%, z → 0.0740%}
```

Aqui está um exemplo do que obtemos se gerarmos uma sequência de letras com essas probabilidades:

```
rronoitadatcaeaesaotdoysaroiyiinnbantoioestlhddeocneooewceseciselnodrtrdgriscsatsepesdcnio
    uhoetsedeyhedslernevstothindtbmnaohngotannbthrdthtonsipieldn
```

Podemos dividir isso em "palavras" adicionando espaços como se fossem letras, com uma certa probabilidade:

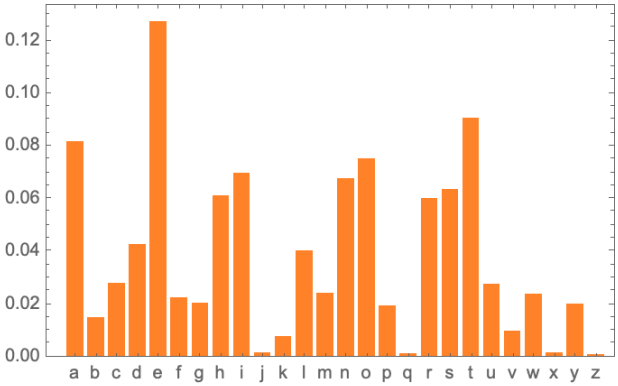
```
sd n oeiaim satnwhoo eer rtr ofiianordrenapwokom del oaas ill e h f
    rellptohltvoettseodtrncilntehtotrkthrslo hdaol n sriaefr hthehtn ld gpod a  h y oi
```

Podemos melhorar um pouco a formação de "palavras" forçando a distribuição dos "comprimentos das palavras" a corresponder à distribuição em inglês:

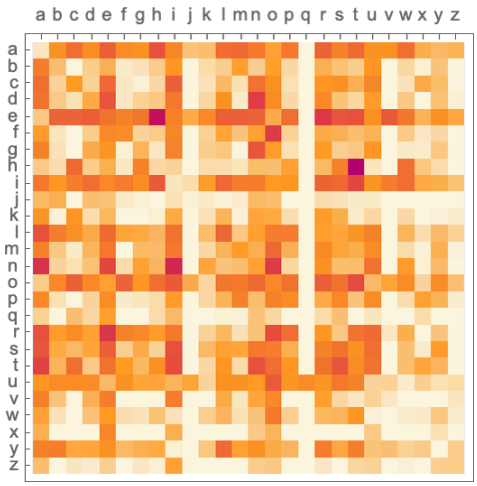
nlu oe nusaetat llfo oeme rrhrtn xdses ohm oa tne ebedcon oarvthv ist

Não conseguimos formar nenhuma "palavra propriamente dita" aqui, mas os resultados parecem um pouco melhores. Para avançarmos, porém, precisamos fazer mais do que simplesmente escolher cada letra aleatoriamente. E, por exemplo, sabemos que se tivermos um "q", a próxima letra basicamente tem que ser "u".

Aqui está um gráfico das probabilidades de cada letra individualmente:



E aqui está um gráfico que mostra as probabilidades de pares de letras ("2-gramas") em um texto típico em inglês. As possíveis primeiras letras são mostradas na horizontal e as segundas letras na vertical:



E vemos aqui, por exemplo, que a coluna “q” está em branco (probabilidade zero), exceto na linha “u”. Certo, então agora, em vez de gerar nossas “palavras” letra por letra, vamos gerá-las observando duas letras por vez, usando essas probabilidades de “2-gramas”. Aqui está um exemplo do resultado — que por acaso inclui algumas “palavras reais”:

on inguman men ise forerenoft weat iofobato buc ous corew ousesetiv
falle tinouco ryefo ra the ecederi pasuthrgr cuconom tra tesla wil tat pere thi

Com uma quantidade suficiente de texto em inglês, podemos obter estimativas bastante precisas não apenas para as probabilidades de letras individuais ou pares de letras (2-gramas), mas também para sequências maiores de letras. E se gerarmos "palavras

elas se tornam progressivamente "mais realistas":

0	on gxeeetowmt tsifhy ah aufnsoc ior oia itlt bnc tu ih uls
1	ri io os ot timumumoi gymyestit ate bshe abol viowr wotybeat mecho
2	wore hi usinallistin hia ale warou pothe of premetra bect upo pr
3	qual musin was witherins wil por vie surgedygua was suchinguary outheydays theresist
4	stud made yello adenced through theirs from cent intous wherefo proteined screa
5	special average vocab consumer market prepara injury trade consa usually speci utility

Mas vamos agora assumir — mais ou menos como o ChatGPT faz — que estamos lidando com palavras inteiras, não com letras. Existem cerca de 40.000 palavras de uso razoavelmente comum em inglês . E, ao analisar um grande conjunto de textos em inglês (digamos, alguns milhões de livros, com um total de algumas centenas de bilhões de palavras), podemos obter uma estimativa da frequência de cada palavra . E, usando isso, podemos começar a gerar "frases", nas quais cada palavra é escolhida aleatoriamente e de forma independente, com a mesma probabilidade de aparecer no conjunto de textos. Aqui está um exemplo do que obtemos:

```
of program excessive been by was research rate not here of of other is men
    were against are show they the different the half the the in any were leaved
```

Como era de se esperar, isso não faz sentido. Então, como podemos melhorar? Assim como fazemos com as letras, podemos começar a levar em conta não apenas as probabilidades de palavras individuais, mas também as probabilidades de pares ou *n*-gramas de palavras. Fazendo isso para pares, aqui estão 5 exemplos do que obtemos, em todos os casos começando com a palavra "gato":

cat through shipping variety is made the aid emergency can the
cat for the book flip was generally decided to design of
cat at safety to contain the vicinity coupled between electric public
cat throughout in a confirmation procedure and two were difficult music
cat on the theory an already from a representation before a

Está ficando um pouco mais "sensato". E podemos imaginar que, se conseguíssemos usar n -gramas suficientemente longos, basicamente "obteríamos um ChatGPT" — no sentido de que obteríamos algo que geraria sequências de palavras do tamanho de uma redação com as "probabilidades gerais corretas para uma redação". Mas aqui está o problema: simplesmente não existe texto em inglês suficiente já escrito para que possamos deduzir essas probabilidades.

Numa busca rápida na internet, podem existir algumas centenas de bilhões de palavras; em livros digitalizados, podem existir outras cem bilhões. Mas com 40.000 palavras comuns, mesmo o número de possíveis 2-gramas já chega a 1,6 bilhão — e o número de

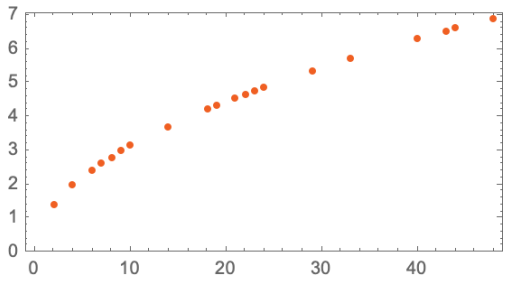
probabilidades, mesmo para todas essas possibilidades, a partir dos textos disponíveis. E quando chegamos a "fragmentos de ensaio" de 20 palavras, o número de possibilidades é maior do que o número de partículas no universo, então, em certo sentido, jamais poderíamos escrevê-las todas.

Então, o que podemos fazer? A grande ideia é criar um modelo que nos permita estimar as probabilidades de ocorrência de determinadas sequências, mesmo que nunca tenhamos visto explicitamente essas sequências no conjunto de textos que analisamos. E no cerne do ChatGPT está justamente um chamado "modelo de linguagem de grande porte" (LLM, na sigla em inglês), desenvolvido para estimar essas probabilidades com precisão.

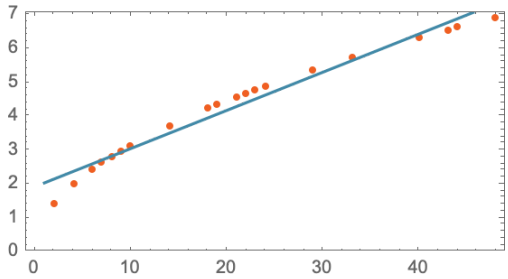
O que é um modelo?

Imagine que você queira saber (como Galileu fez no final do século XVI) quanto tempo uma bala de canhão, lançada de cada andar da Torre de Pisa, levaria para atingir o solo. Bem, você poderia simplesmente medir em cada caso e fazer uma tabela com os resultados. Ou você poderia fazer o que é a essência da ciência teórica: criar um modelo que forneça algum tipo de procedimento para calcular a resposta, em vez de apenas medir e memorizar cada caso.

Vamos imaginar que temos dados (um tanto idealizados) sobre quanto tempo a bala de canhão leva para cair de vários andares:

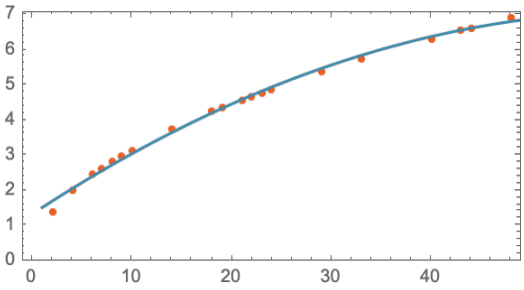


Como podemos calcular quanto tempo levará para cair de um piso sobre o qual não temos dados explícitos? Nesse caso específico, podemos usar as leis da física conhecidas para fazer o cálculo. Mas digamos que tudo o que temos são os dados, e não sabemos quais leis subjacentes os regem. Então, poderíamos fazer uma estimativa matemática, como, por exemplo, usar uma linha reta como modelo:

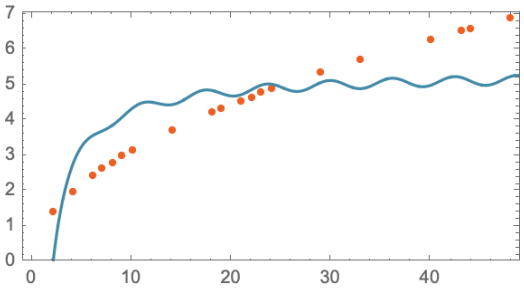


aproxima dos dados que nos foram fornecidos. E a partir dessa linha reta, podemos estimar o tempo de queda para qualquer andar.

Como sabíamos que deveríamos usar uma linha reta aqui? Em certo nível, não sabíamos. É simplesmente algo matematicamente simples, e estamos acostumados ao fato de que muitos dados que medimos se ajustam bem a equações matematicamente simples. Poderíamos tentar algo matematicamente mais complexo — digamos $a + bx + cx^2 - e$, nesse caso, teríamos um resultado melhor *:*



As coisas podem dar muito errado, no entanto. Veja, por exemplo, o melhor que podemos fazer com $a + b / x + c \sin(x)$:



É importante entender que nunca existe um "modelo sem modelo". Qualquer modelo que você use tem uma estrutura subjacente específica — e um conjunto de "parâmetros que você pode ajustar" para que ele se ajuste aos seus dados. No caso do ChatGPT, muitos desses "parâmetros" são usados — na verdade, 175 bilhões deles.

Mas o mais notável é que a estrutura subjacente do ChatGPT — com "apenas" essa quantidade de parâmetros — é suficiente para criar um modelo que calcula as probabilidades da próxima palavra "bem o suficiente" para nos fornecer textos com tamanho razoável de ensaio.

Modelos para tarefas semelhantes às humanas

O exemplo que demos acima envolve a criação de um modelo para dados numéricos que essencialmente provêm da física simples — área na qual sabemos há vários séculos que a "matemática simples se aplica". Mas, para o ChatGPT, precisamos criar um modelo de texto em linguagem humana, do tipo produzido pelo cérebro humano. E

simples". Então, como seria um modelo para isso?

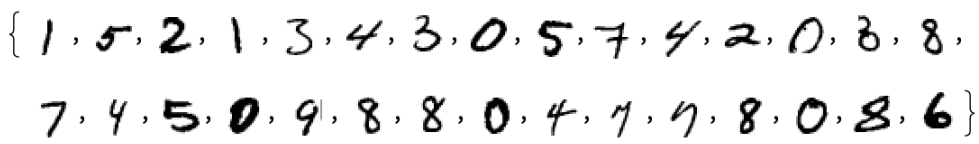
Antes de falarmos sobre linguagem, vamos abordar outra tarefa semelhante à humana: o reconhecimento de imagens. E como um exemplo simples disso, vamos considerar imagens de dígitos (e, sim, este é um exemplo clássico de aprendizado de máquina):



Uma coisa que poderíamos fazer é obter várias imagens de amostra para cada dígito:



Para descobrir se uma imagem que recebemos como entrada corresponde a um dígito específico, poderíamos simplesmente fazer uma comparação explícita pixel a pixel com as amostras que temos. Mas, como humanos, certamente fazemos algo melhor — porque ainda conseguimos reconhecer dígitos, mesmo quando são manuscritos, por exemplo, e apresentam todos os tipos de modificações e distorções:



Ao criarmos um modelo para nossos dados numéricos acima, conseguimos pegar um valor numérico x que nos foi fornecido e simplesmente calcular $a + bx$ para valores específicos *de* a e b . Então, se tratarmos o valor de nível de cinza de cada pixel como uma variável x _i , existe alguma função de todas essas variáveis que — quando avaliada — nos diga a qual dígito a imagem pertence? Acontece que é possível construir tal função. Não surpreendentemente, porém, não é particularmente simples. E um exemplo típico poderia envolver talvez meio milhão de operações matemáticas.

Mas o resultado final é que, se alimentarmos essa função com o conjunto de valores de pixels de uma imagem, obteremos o número que especifica a qual dígito temos a imagem. Mais tarde, falaremos sobre como essa função pode ser construída e sobre a ideia de redes neurais. Mas, por enquanto, vamos tratar a função como uma caixa preta, onde inserimos imagens de, digamos, dígitos manuscritos (como matrizes de valores de pixels) e obtemos os números correspondentes:

```
In[ ]:= NetModel["..."][{ 7, 0, 9, 7, 8, 2, 4, 1, 1, 1 }]

Out[ ]= {7, 0, 9, 7, 8, 2, 4, 1, 1, 1}
```

progressivamente. Por um breve período, nossa função ainda o "reconhece", neste caso, como um "2". Mas logo ela o "perde" e começa a dar o resultado "errado":

```
In[ ]:= NetModel["..."][{2, 2, 2, 2, 2, 2, 2, 2, 2}]
Out[ ]= {2, 2, 2, 1, 1, 1, 1, 1, 1}
```

Mas por que dizemos que é o resultado “errado”? Nesse caso, sabemos que obtivemos todas as imagens desfocando um “2”. Mas se nosso objetivo é produzir um modelo do que os humanos podem fazer ao reconhecer imagens, a verdadeira questão a ser feita é o que um humano teria feito se apresentado a uma dessas imagens desfocadas, sem saber de onde ela veio.

E temos um “bom modelo” se os resultados que obtemos da nossa função normalmente concordarem com o que um humano diria. E o fato científico não trivial é que, para uma tarefa de reconhecimento de imagem como esta, agora sabemos basicamente como construir funções que fazem isso.

Podemos "provar matematicamente" que elas funcionam? Bem, não. Porque para isso precisaríamos de uma teoria matemática do que nós, humanos, fazemos. Pegue a imagem do "2" e altere alguns pixels. Podemos imaginar que, com apenas alguns pixels "fora do lugar", ainda deveríamos considerar a imagem como um "2". Mas até que ponto isso deveria ir? É uma questão de percepção visual humana . E, sim, a resposta sem dúvida seria diferente para abelhas ou polvos — e potencialmente completamente diferente para supostos extraterrestres .

Redes Neurais

Certo, então como funcionam, na prática, nossos modelos típicos para tarefas como reconhecimento de imagem ? A abordagem atual mais popular — e bem-sucedida — utiliza redes neurais . Inventadas — em uma forma notavelmente próxima ao seu uso atual — na década de 1940 , as redes neurais podem ser consideradas idealizações simples de como o cérebro parece funcionar .

No cérebro humano existem cerca de 100 bilhões de neurônios (células nervosas), cada um capaz de produzir um pulso elétrico até talvez mil vezes por segundo. Os neurônios estão conectados em uma rede complexa, com cada neurônio possuindo ramificações que lhe permitem transmitir sinais elétricos para talvez milhares de outros neurônios. E, em uma aproximação grosseira, o fato de um determinado neurônio produzir um pulso elétrico em um dado momento depende dos pulsos que ele recebe de outros neurônios — com diferentes conexões contribuindo com diferentes "pesos".

Quando “vemos uma imagem”, o que acontece é que, quando os fótons de luz da imagem incidem sobre as células (“fotorreceptoras”) na parte posterior dos nossos

conectadas a outras células nervosas e, eventualmente, os sinais percorrem toda uma sequência de camadas de neurônios. E é nesse processo que “reconhecemos” a imagem, eventualmente “formando o pensamento” de que estamos “vendo um 2” (e talvez, no final, fazendo algo como dizer a palavra “dois” em voz alta).

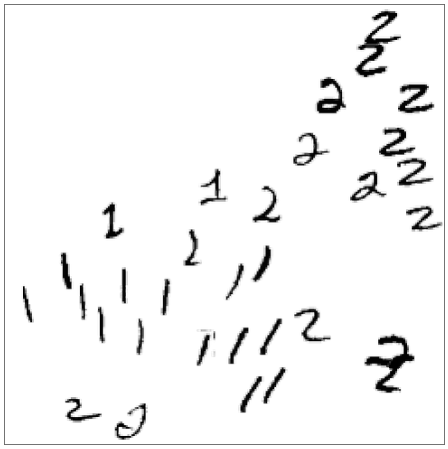
A função de "caixa preta" da seção anterior é uma versão "matematizada" de uma rede neural desse tipo. Ela possui 11 camadas (embora apenas 4 sejam "camadas principais"):

NetChain

		image
	Input	array (size: 1×28×28)
1	ConvolutionLayer	array (size: 20×24×24)
2	Ramp	array (size: 20×24×24)
3	PoolingLayer	array (size: 20×12×12)
4	ConvolutionLayer	array (size: 50×8×8)
5	Ramp	array (size: 50×8×8)
6	PoolingLayer	array (size: 50×4×4)
7	FlattenLayer	vector (size: 800)
8	LinearLayer	vector (size: 500)
9	Ramp	vector (size: 500)
10	LinearLayer	vector (size: 10)
11	SoftmaxLayer	vector (size: 10)
	Output	class

Não há nada particularmente "derivado da teoria" nessa rede neural; é simplesmente algo que — em 1998 — foi construído como um projeto de engenharia e que se mostrou funcional. (Claro, isso não é muito diferente de como poderíamos descrever nossos cérebros como tendo sido produzidos pelo processo de evolução biológica.)

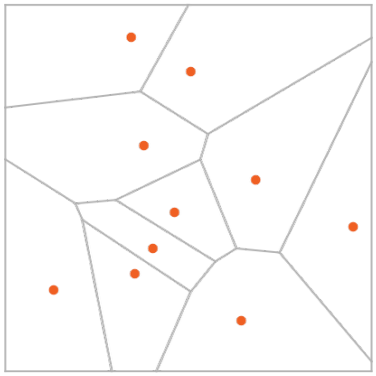
Certo, mas como uma rede neural como essa "reconhece coisas"? A chave está na noção de atratores . Imagine que temos imagens manuscritas de 1 e 2:



De alguma forma, queremos que todos os "1s" sejam "atraídos para um lugar" e todos os "2s" sejam "atraídos para outro lugar". Ou, dito de outra forma, se uma imagem está de alguma forma " mais próxima de ser um 1 " do que de ser um 2, queremos que ela acabe no "lugar 1" e vice-versa.

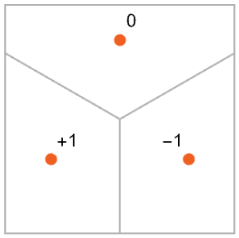
Para simplificar a analogia, imagine que temos certas posições no plano, indicadas por pontos (em um cenário real, poderiam ser a localização de cafeterias). Podemos

ponto mais próximo (ou seja, sempre iríamos à cafeteria mais próxima). Podemos representar isso dividindo o plano em regiões (“bacias de atração”) separadas por “divisores de águas” idealizados:

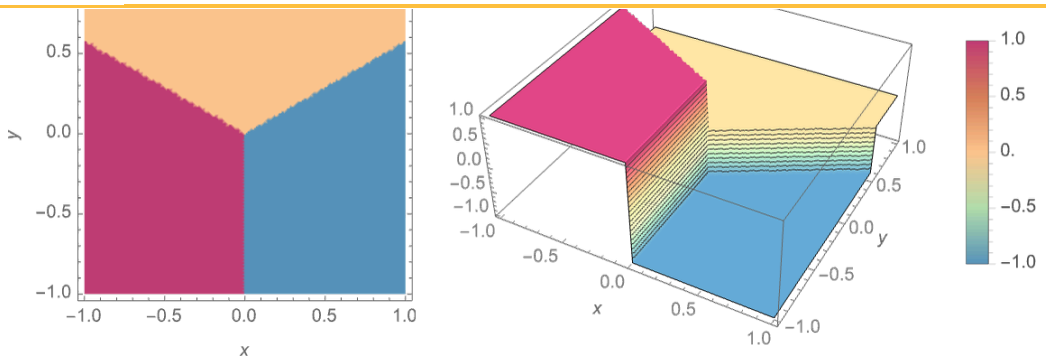


Podemos pensar nisso como a implementação de uma espécie de "tarefa de reconhecimento" na qual não estamos identificando qual dígito uma determinada imagem "se parece mais" — mas sim, de forma bem direta, observando a qual ponto um determinado ponto está mais próximo. (A configuração do "diagrama de Voronoi" que estamos mostrando aqui separa pontos em um espaço euclidiano 2D; a tarefa de reconhecimento de dígitos pode ser vista como algo muito semelhante — mas em um espaço de 784 dimensões formado pelos níveis de cinza de todos os pixels em cada imagem.)

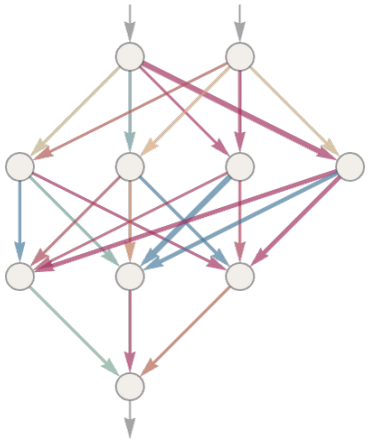
Então, como fazemos uma rede neural "executar uma tarefa de reconhecimento"? Vamos considerar este caso muito simples:



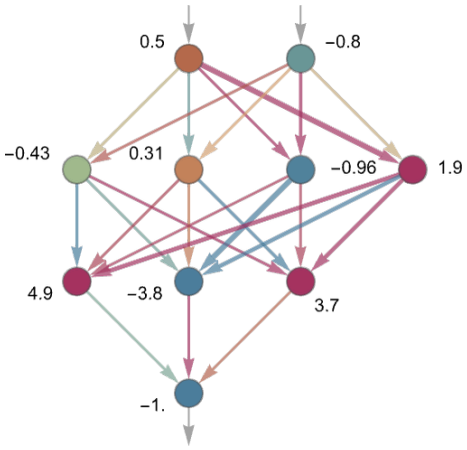
Nosso objetivo é receber uma “entrada” correspondente a uma posição $\{ x , y \}$ e então “reconhecê-la” como o ponto mais próximo dentre os três. Ou, em outras palavras, queremos que a rede neural calcule uma função de $\{ x , y \}$ como:



Então, como fazemos isso com uma rede neural? Em última análise, uma rede neural é uma coleção interconectada de "neurônios" idealizados — geralmente organizados em camadas — sendo um exemplo simples:



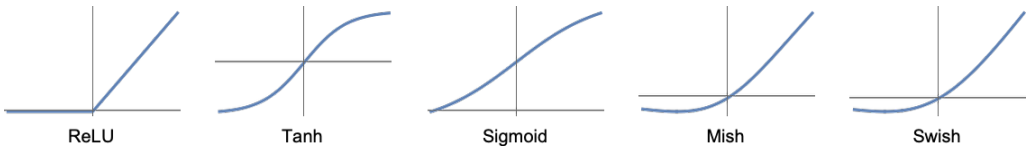
Cada "neurônio" é configurado para avaliar uma função numérica simples. E para "usar" a rede, basta inserir números (como nossas coordenadas x e y) na camada superior, e então os neurônios em cada camada "avaliam suas funções" e propagam os resultados pela rede, produzindo, por fim, o resultado final na camada inferior.



Na configuração tradicional (inspirada biologicamente), cada neurônio possui efetivamente um conjunto específico de "conexões de entrada" provenientes dos neurônios da camada anterior, sendo atribuído a cada conexão um determinado "peso" (que pode ser um número positivo ou negativo). O valor de um neurônio específico é determinado multiplicando-se os valores dos "neurônios anteriores" por seus

finalmente, aplicando-se uma função de "limiarização" (ou "ativação"). Em termos matemáticos, se um neurônio possui entradas $x = \{x_1, x_2, ...\}$, então calculamos $f[w . x + b]$, onde os pesos w e a constante b são geralmente escolhidos de forma diferente para cada neurônio na rede; a função f, por sua vez, costuma ser a mesma.

Calcular $w . x + b$ é simplesmente uma questão de multiplicação e adição de matrizes. A "função de ativação" f introduz a não linearidade (e, em última análise, é o que leva a um comportamento não trivial). Várias funções de ativação são comumente usadas; aqui, usaremos apenas a Ramp (ou ReLU):



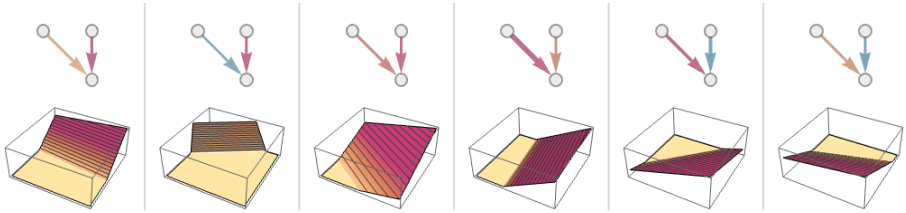
Para cada tarefa que queremos que a rede neural execute (ou, equivalentemente, para cada função geral que queremos que ela avalie), teremos diferentes opções de pesos. (E — como discutiremos mais adiante — esses pesos são normalmente determinados pelo "treinamento" da rede neural usando aprendizado de máquina a partir de exemplos das saídas desejadas.)

Em última análise, toda rede neural corresponde a alguma função matemática geral — embora possa ser complicado descrevê-la. Para o exemplo acima, seria:

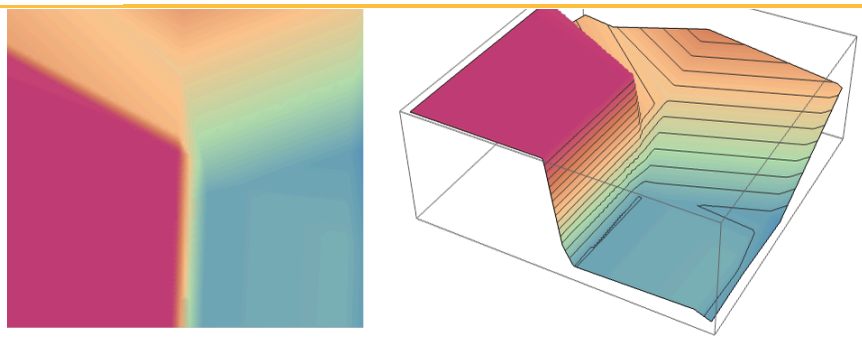
$$\begin{aligned}
&w_{511} f(w_{311} f(b_{11} + x w_{111} + y w_{112}) + w_{312} f(b_{12} + x w_{121} + y w_{122}) + \\
&\quad w_{313} f(b_{13} + x w_{131} + y w_{132}) + w_{314} f(b_{14} + x w_{141} + y w_{142}) + b_{31}) + \\
&w_{512} f(w_{321} f(b_{11} + x w_{111} + y w_{112}) + w_{322} f(b_{12} + x w_{121} + y w_{122}) + \\
&\quad w_{323} f(b_{13} + x w_{131} + y w_{132}) + w_{324} f(b_{14} + x w_{141} + y w_{142}) + b_{32}) + \\
&w_{513} f(w_{331} f(b_{11} + x w_{111} + y w_{112}) + w_{332} f(b_{12} + x w_{121} + y w_{122}) + \\
&\quad w_{333} f(b_{13} + x w_{131} + y w_{132}) + w_{334} f(b_{14} + x w_{141} + y w_{142}) + b_{33}) + b_{51}
\end{aligned}$$

A rede neural do ChatGPT também corresponde a uma função matemática como esta, mas efetivamente com bilhões de termos.

Mas vamos voltar aos neurônios individuais. Aqui estão alguns exemplos das funções que um neurônio com duas entradas (representando as coordenadas x e y) pode computar com diferentes escolhas de pesos e constantes (e com a função de ativação Ramp):

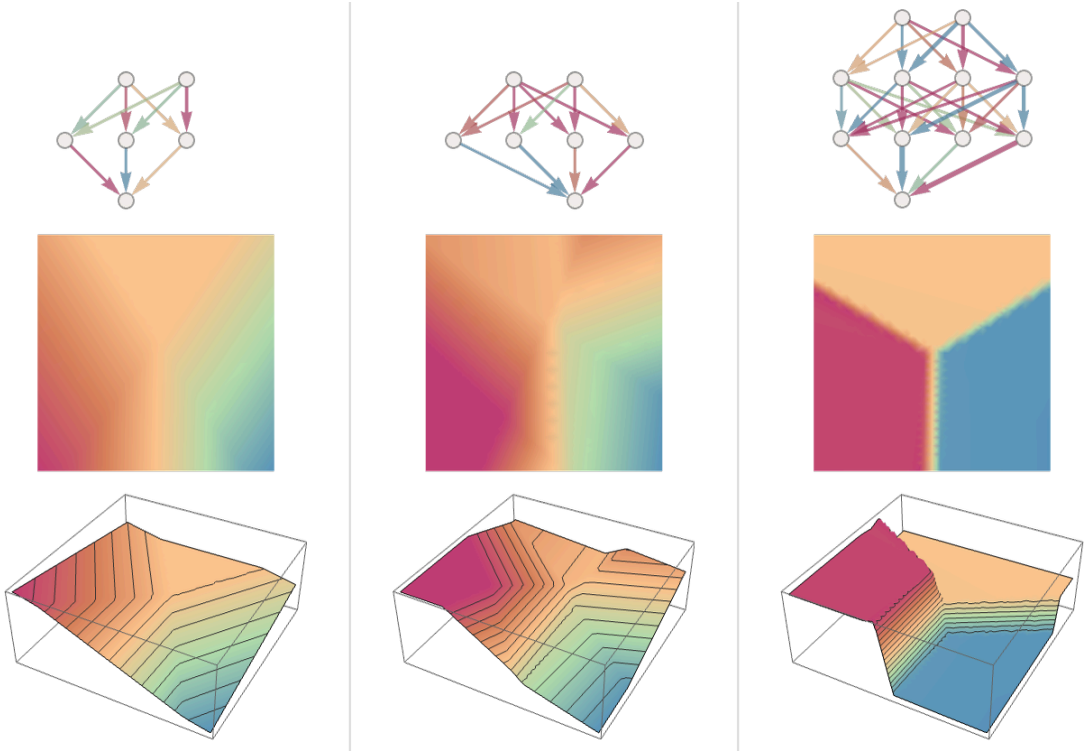


Mas e quanto à rede maior vista de cima? Bem, aqui está o que ela calcula:



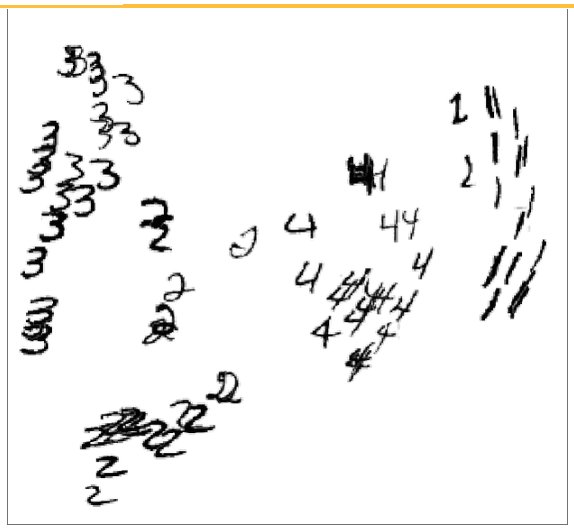
Não está exatamente "certo", mas está próximo da função de "ponto mais próximo" que mostramos acima.

Vejamos o que acontece com outras redes neurais. Em cada caso, como explicaremos mais adiante, estamos usando aprendizado de máquina para encontrar a melhor escolha de pesos. Em seguida, mostramos aqui o que a rede neural com esses pesos calcula:



Redes maiores geralmente são mais eficientes em aproximar a função que buscamos. E no "centro de cada bacia de atração", normalmente obtemos exatamente a resposta desejada. Mas nas extremidades — onde a rede neural "tem dificuldade em se decidir" — as coisas podem ficar mais complicadas.

Com essa tarefa simples de reconhecimento matemático, a resposta correta fica clara. Mas no problema de reconhecer dígitos manuscritos, não é tão óbvio. E se alguém escrevesse um "2" tão mal que parecesse um "7", etc.? Mesmo assim, podemos perguntar como uma rede neural distingue dígitos — e isso nos dá uma pista:



Podemos dizer "matematicamente" como a rede faz suas distinções? Na verdade, não. Ela simplesmente "faz o que a rede neural faz". Mas acontece que isso normalmente parece concordar bastante bem com as distinções que nós, humanos, fazemos.

Vamos considerar um exemplo mais elaborado. Digamos que temos imagens de gatos e cachorros. E temos uma rede neural treinada para distingui-los . Veja o que ela poderia fazer com alguns exemplos:



Agora, a questão da "resposta certa" está ainda menos clara. E quanto a um cachorro vestido com uma fantasia de gato? Etc. Qualquer que seja a entrada fornecida, a rede neural gerará uma resposta, e de uma forma razoavelmente consistente com a maneira como os humanos pensariam. Como mencionei anteriormente, isso não é um fato que podemos "derivar de princípios fundamentais". É apenas algo que foi comprovado empiricamente, pelo menos em certos domínios. Mas essa é uma razão fundamental pela qual as redes neurais são úteis: elas capturam, de alguma forma, uma maneira "humana" de fazer as coisas.

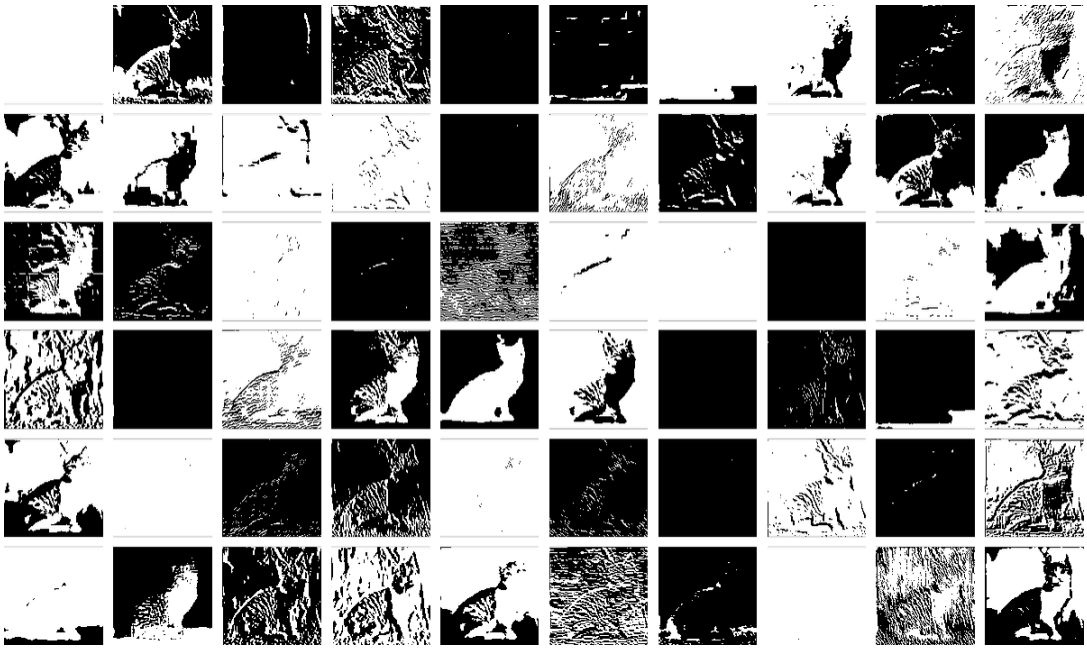
você comece dizendo: "Bem, eu vejo suas orelhas pontudas, etc." Mas não é muito fácil explicar como você reconheceu a imagem como sendo de um gato. É simplesmente porque, de alguma forma, seu cérebro chegou a essa conclusão. Mas, para um cérebro, não há como (pelo menos ainda) "entrar" e ver como ele chegou a essa conclusão. E quanto a uma rede neural (artificial)? Bem, é simples ver o que cada "neurônio" faz quando você mostra a imagem de um gato. Mas mesmo obter uma visualização básica geralmente é muito difícil.

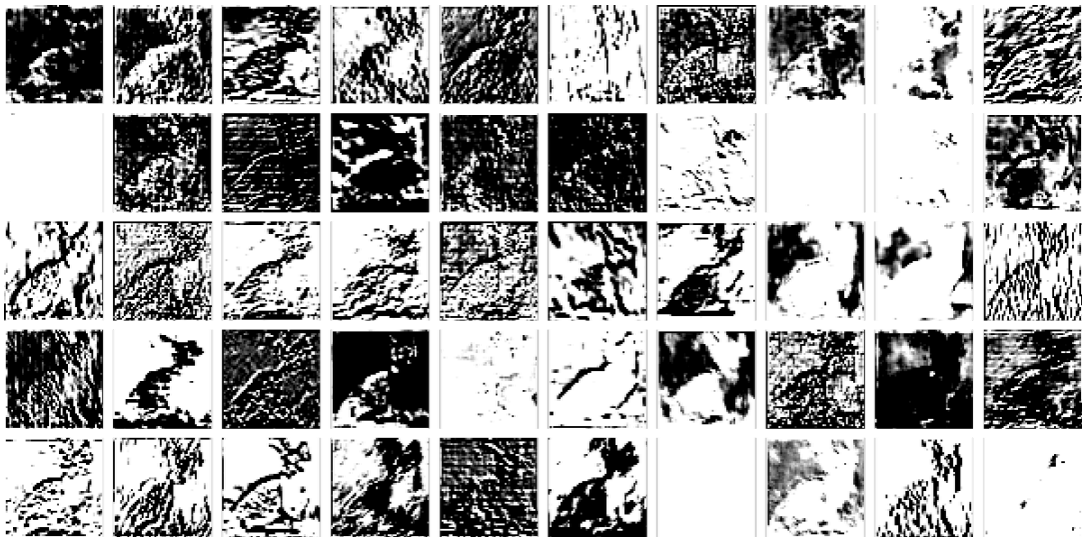
Na rede neural final que usamos para o problema do "ponto mais próximo" acima, existem 17 neurônios. Na rede para reconhecimento de dígitos manuscritos, existem 2190. E na rede que estamos usando para reconhecer gatos e cachorros, existem 60.650. Normalmente, seria bastante difícil visualizar o que equivale a um espaço de 60.650 dimensões. Mas, como esta é uma rede configurada para lidar com imagens, muitas de suas camadas de neurônios são organizadas em matrizes, como as matrizes de pixels que ela está analisando.

E se considerarmos a imagem típica de um gato?



então podemos representar os estados dos neurônios na primeira camada por uma coleção de imagens derivadas — muitas das quais podemos interpretar facilmente como sendo coisas como “o gato sem o fundo” ou “o contorno do gato”:





Mas, em geral, podemos dizer que a rede neural está "selecionando certas características" (talvez orelhas pontudas estejam entre elas) e usando-as para determinar o que a imagem representa. Mas será que essas características têm nomes específicos — como "orelhas pontudas"? Na maioria das vezes, não.

Será que nossos cérebros usam características semelhantes? Na maioria dos casos, não sabemos. Mas é notável que as primeiras camadas de uma rede neural como a que estamos mostrando aqui parecem selecionar aspectos das imagens (como as bordas dos objetos) que parecem ser semelhantes aos que sabemos serem selecionados pelo primeiro nível de processamento visual no cérebro.

Mas digamos que queremos uma “teoria de reconhecimento de gatos” em redes neurais. Podemos dizer: “Vejam, esta rede em particular faz isso” — e isso imediatamente nos dá uma ideia da “dificuldade do problema” (e, por exemplo, quantos neurônios ou camadas podem ser necessários). Mas, pelo menos por enquanto, não temos como “fornecer uma descrição narrativa” do que a rede está fazendo. E talvez isso aconteça porque ela é realmente irredutível computacionalmente, e não há uma maneira geral de descobrir o que ela faz, exceto rastreando explicitamente cada etapa. Ou talvez seja apenas que ainda não “desvendamos a ciência” e identificamos as “leis naturais” que nos permitem resumir o que está acontecendo.

Encontraremos os mesmos tipos de problemas quando falarmos sobre a geração de linguagem com o ChatGPT. E, novamente, não está claro se existem maneiras de "resumir o que ele está fazendo". Mas a riqueza e os detalhes da linguagem (e nossa experiência com ela) podem nos permitir ir mais longe do que com imagens.

Aprendizado de Máquina e o Treinamento de Redes Neurais

Até agora, falamos sobre redes neurais que "já sabem" como realizar tarefas específicas. Mas o que torna as redes neurais tão úteis (presumivelmente também no cérebro) é que

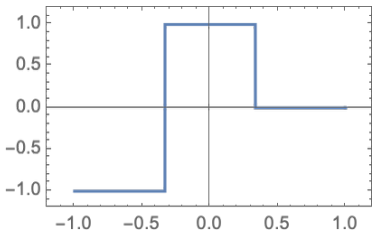
podem ser "treinadas a partir de exemplos" de forma incremental para realizar essas tarefas.

Quando criamos uma rede neural para distinguir gatos de cachorros, não precisamos escrever um programa que (digamos) encontre explicitamente os bigodes; em vez disso, mostramos muitos exemplos do que é um gato e do que é um cachorro, e então deixamos a rede "aprender por máquina" com esses exemplos como distingui-los.

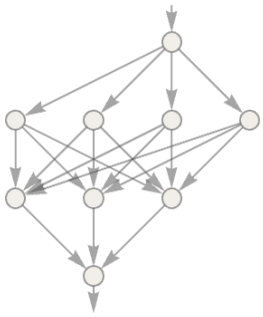
E a questão é que a rede treinada "generaliza" a partir dos exemplos específicos que lhe são mostrados. Assim como vimos acima, não se trata simplesmente de a rede reconhecer o padrão de pixels específico de uma imagem de gato que lhe foi apresentada; trata-se, na verdade, de a rede neural conseguir distinguir imagens com base no que consideramos ser uma espécie de "característica geral de um gato".

Então, como funciona o treinamento de redes neurais na prática? Essencialmente, o que sempre tentamos fazer é encontrar pesos que permitam à rede neural reproduzir com sucesso os exemplos que fornecemos. E então, contamos com a rede neural para "interpolar" (ou "generalizar") "entre" esses exemplos de uma maneira "razoável".

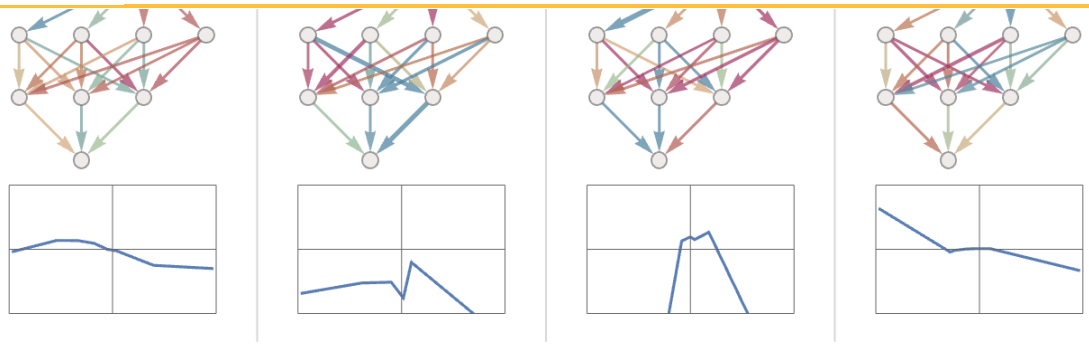
Vamos analisar um problema ainda mais simples do que o do ponto mais próximo acima. Vamos tentar fazer com que uma rede neural aprenda a função:



Para esta tarefa, precisaremos de uma rede que tenha apenas uma entrada e uma saída, como:

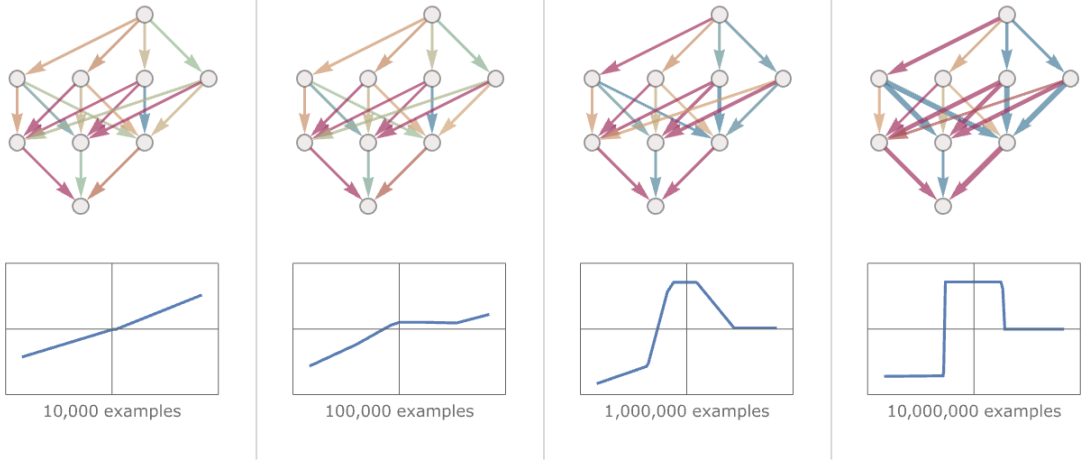


Mas quais pesos, etc., devemos usar? Com cada conjunto possível de pesos, a rede neural calculará alguma função. E, por exemplo, veja o que ela faz com alguns conjuntos de pesos escolhidos aleatoriamente:



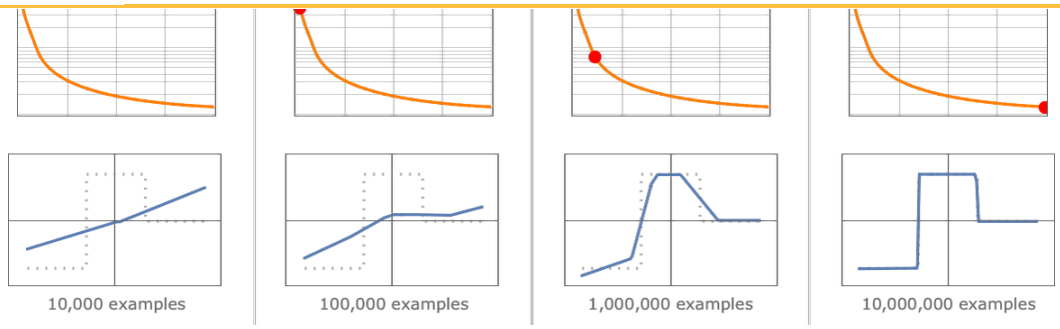
E, sim, podemos ver claramente que em nenhum desses casos chega perto de reproduzir a função que desejamos. Então, como encontramos pesos que reproduzam a função?

A ideia básica é fornecer muitos exemplos de "entrada → saída" para "aprender" — e então tentar encontrar pesos que reproduzam esses exemplos. Aqui está o resultado de fazer isso com um número progressivamente maior de exemplos:



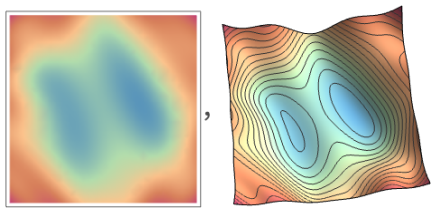
Em cada etapa desse “treinamento”, os pesos da rede são ajustados progressivamente — e vemos que, eventualmente, obtemos uma rede que reproduz com sucesso a função desejada. Então, como ajustamos os pesos? A ideia básica é, em cada etapa, verificar “quão perto estamos” de obter a função desejada — e então atualizar os pesos de forma a nos aproximarmos dela.

Para descobrir "quão longe estamos", calculamos o que geralmente é chamado de "função de perda" (ou, às vezes, "função de custo"). Aqui, estamos usando uma função de perda simples (L2), que é apenas a soma dos quadrados das diferenças entre os valores que obtemos e os valores reais. E o que vemos é que, à medida que nosso processo de treinamento progride, a função de perda diminui progressivamente (seguindo uma certa "curva de aprendizado" que é diferente para diferentes tarefas) — até atingirmos um ponto em que a rede (pelo menos com boa aproximação) reproduz com sucesso a função que desejamos:

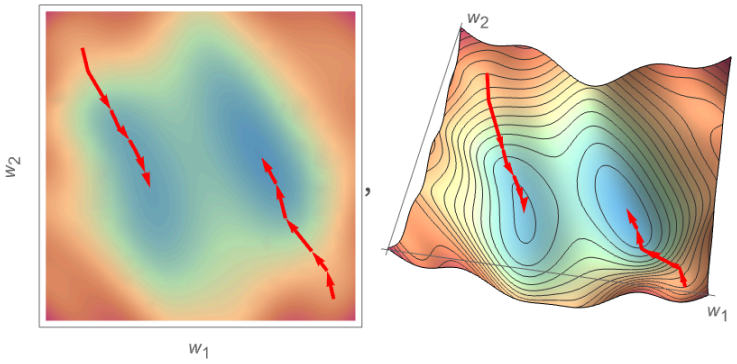


Muito bem, então a última parte essencial a explicar é como os pesos são ajustados para reduzir a função de perda. Como já dissemos, a função de perda nos dá uma "distância" entre os valores que obtivemos e os valores reais. Mas os "valores que obtivemos" são determinados em cada etapa pela versão atual da rede neural — e pelos pesos nela presentes. Agora imagine que os pesos sejam variáveis — digamos, w_i. Queremos descobrir como ajustar os valores dessas variáveis para minimizar a perda que depende delas.

Por exemplo, imagine (numa simplificação incrível das redes neurais típicas usadas na prática) que temos apenas dois pesos, w_1 e w_2. Então, poderíamos ter uma função de perda que, em função de w_1 e w_2, se parece com isto:



A análise numérica oferece diversas técnicas para encontrar o mínimo em casos como este. Mas uma abordagem típica consiste em seguir progressivamente o caminho de maior declive a partir de quaisquer valores anteriores de w_1 e w_2 que tivéssemos:



Tal como a água que desce uma montanha, tudo o que se garante é que este processo acabará por atingir um mínimo local na superfície ("um lago de montanha"); pode muito bem não atingir o mínimo global final.

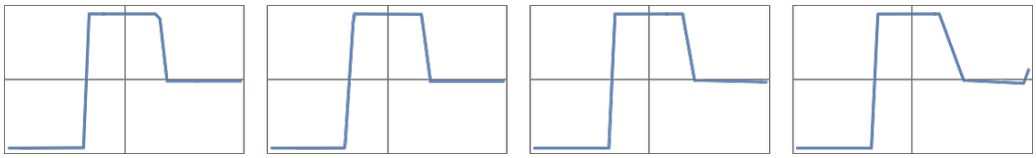
Não é óbvio que seja viável encontrar o caminho de maior declive na "paisagem de pesos". Mas o cálculo vem em nosso auxílio. Como mencionado anteriormente,

matemática — que depende de suas entradas e de seus pesos. Mas agora considere a diferenciação em relação a esses pesos. Acontece que a regra da cadeia do cálculo nos permite, na prática, "desvendar" as operações realizadas pelas camadas sucessivas da rede neural. E o resultado é que podemos — pelo menos em alguma aproximação local — "inverter" a operação da rede neural e encontrar progressivamente pesos que minimizem a perda associada à saída.

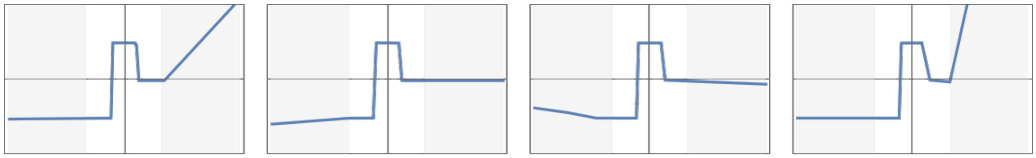
A imagem acima mostra o tipo de minimização que poderíamos precisar fazer no caso irrealisticamente simples de apenas 2 pesos. Mas acontece que mesmo com muito mais pesos (o ChatGPT usa 175 bilhões) ainda é possível realizar a minimização, pelo menos com algum nível de aproximação. E, de fato, o grande avanço no "aprendizado profundo" que ocorreu por volta de 2011 esteve associado à descoberta de que, de certa forma, pode ser mais fácil realizar (pelo menos aproximar) a minimização quando há muitos pesos envolvidos do que quando há poucos.

Em outras palavras — de forma um tanto contraintuitiva — pode ser mais fácil resolver problemas mais complexos com redes neurais do que problemas mais simples. E a razão geral para isso parece ser que, quando se tem muitas "variáveis de peso", tem-se um espaço de alta dimensionalidade com "muitas direções diferentes" que podem levar ao mínimo — enquanto que, com menos variáveis, é mais fácil ficar preso em um mínimo local ("lago na montanha") do qual não há "direção para sair".

Vale ressaltar que, em casos típicos, existem muitos conjuntos diferentes de pesos que resultarão em redes neurais com desempenho praticamente idêntico. E, geralmente, no treinamento prático de redes neurais, muitas escolhas aleatórias são feitas, levando a "soluções diferentes, porém equivalentes", como estas:



Mas cada uma dessas “soluções diferentes” terá um comportamento pelo menos ligeiramente diferente. E se pedirmos, por exemplo, uma “extrapolação” para fora da região onde fornecemos os exemplos de treinamento, podemos obter resultados drasticamente diferentes:



Mas qual delas está “certa”? Não há como dizer com certeza. Todas são “consistentes com os dados observados”. Mas todas correspondem a diferentes maneiras “inatas” de

para nós, humanos, do que outras.

A prática e a tradição do treinamento em redes neurais

Particularmente na última década, houve muitos avanços na arte de treinar redes neurais. E, sim, é basicamente uma arte. Às vezes — especialmente em retrospectiva — é possível vislumbrar pelo menos um lampejo de uma "explicação científica" para algo que está sendo feito. Mas, na maioria das vezes, as descobertas foram feitas por tentativa e erro, adicionando ideias e truques que, progressivamente, construíram um conhecimento significativo sobre como trabalhar com redes neurais.

Existem vários pontos-chave. Primeiro, há a questão de qual arquitetura de rede neural deve ser usada para uma tarefa específica. Depois, há a questão crucial de como obter os dados para treinar a rede neural. E, cada vez mais, não se trata de treinar uma rede do zero: em vez disso, uma nova rede pode incorporar diretamente outra rede já treinada ou, pelo menos, usar essa rede para gerar mais exemplos de treinamento para si mesma.

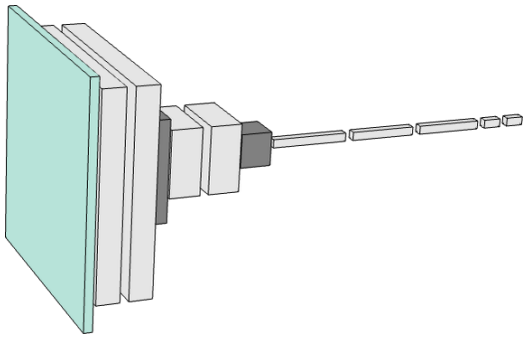
Poderíamos pensar que para cada tipo específico de tarefa seria necessária uma arquitetura de rede neural diferente. Mas o que se descobriu é que a mesma arquitetura muitas vezes funciona mesmo para tarefas aparentemente bastante diferentes. Em certo nível, isso nos lembra a ideia de computação universal (e meu Princípio da Equivalência Computacional), mas, como discutirei mais adiante, acredito que seja mais um reflexo do fato de que as tarefas que normalmente tentamos fazer com que as redes neurais realizem são "semelhantes às humanas" — e as redes neurais podem capturar processos "semelhantes aos humanos" bastante gerais.

Nos primórdios das redes neurais, prevalecia a ideia de que se deveria "fazer a rede neural fazer o mínimo possível". Por exemplo, na conversão de fala em texto, acreditava-se que primeiro se deveria analisar o áudio da fala, dividi-lo em fonemas, etc. Mas o que se descobriu é que — pelo menos para "tarefas semelhantes às humanas" — geralmente é melhor simplesmente tentar treinar a rede neural no "problema de ponta a ponta", deixando que ela "descubra" as características intermediárias necessárias, codificações, etc., por si só.

Havia também a ideia de introduzir componentes individuais complexos na rede neural, para que ela, na prática, "implementasse explicitamente ideias algorítmicas específicas". Mas, mais uma vez, isso se mostrou, em grande parte, inútil; em vez disso, é melhor lidar com componentes muito simples e deixá-los "se organizarem" (embora geralmente de maneiras que não conseguimos entender) para alcançar (presumivelmente) o equivalente a essas ideias algorítmicas.

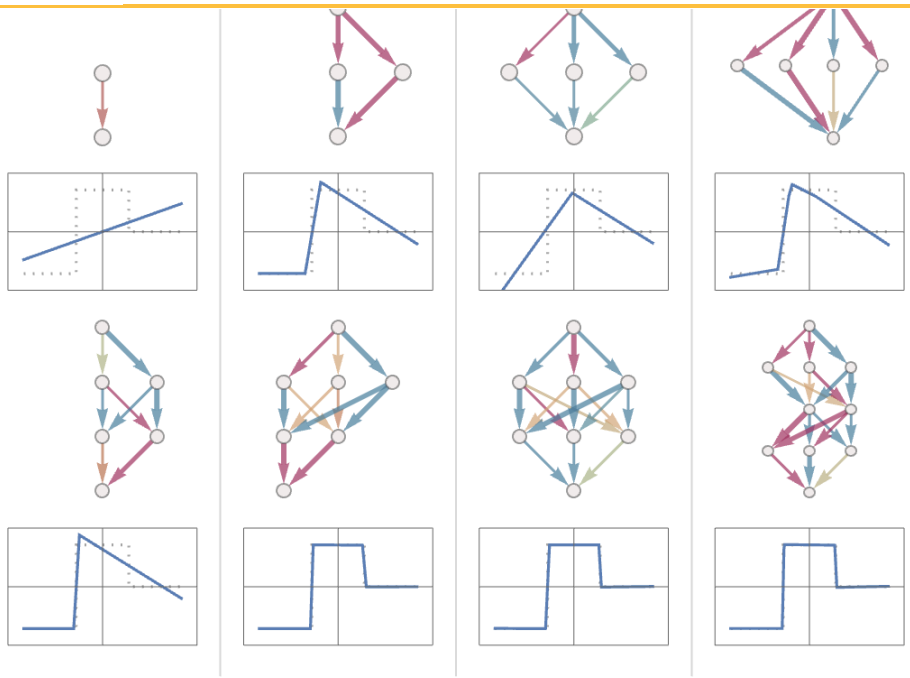
neurais. Assim, por exemplo, ter matrizes 2D de neurônios com conexões locais parece, pelo menos, muito útil nos estágios iniciais do processamento de imagens. E ter padrões de conectividade que se concentram em "analisar sequências" parece útil — como veremos adiante — para lidar com coisas como a linguagem humana, por exemplo, no ChatGPT.

Mas uma característica importante das redes neurais é que — assim como os computadores em geral — elas lidam, em última análise, apenas com dados. E as redes neurais atuais — com as abordagens atuais de treinamento de redes neurais — lidam especificamente com matrizes de números . Mas, durante o processamento, essas matrizes podem ser completamente reorganizadas e remodeladas. Como exemplo, a rede que usamos para identificar dígitos acima começa com uma matriz 2D semelhante a uma imagem, que rapidamente se torna mais densa, com muitos canais, mas depois se concentra em uma matriz 1D que, em última análise, conterá elementos representando os diferentes dígitos de saída possíveis:



Mas, certo, como saber o tamanho ideal de uma rede neural para uma tarefa específica? É uma espécie de arte. Em certo nível, o essencial é saber "quão difícil é a tarefa". Mas, para tarefas que simulam atividades humanas, isso geralmente é muito difícil de estimar. Sim, pode haver uma maneira sistemática de realizar a tarefa de forma muito "mecânica" por computador. Mas é difícil saber se existem o que poderíamos chamar de truques ou atalhos que permitam realizar a tarefa, pelo menos em um nível "semelhante ao humano", com muito mais facilidade. Pode ser necessário enumerar uma árvore de regras gigantesca para jogar um determinado jogo "mecanicamente"; mas pode haver uma maneira muito mais fácil ("heurística") de alcançar um desempenho "em nível humano".

Ao lidar com redes neurais pequenas e tarefas simples, às vezes é possível perceber explicitamente que "não se consegue chegar lá partindo daqui". Por exemplo, veja o melhor resultado que parece ser possível obter na tarefa da seção anterior com algumas pequenas redes neurais:



O que observamos é que, se a rede for muito pequena, ela simplesmente não consegue reproduzir a função desejada. Mas, acima de um certo tamanho, ela não apresenta problemas — pelo menos se for treinada por tempo suficiente, com exemplos suficientes. Aliás, essas imagens ilustram um princípio básico das redes neurais: muitas vezes, é possível usar uma rede menor se houver uma "compressão" no meio que force tudo a passar por um número menor de neurônios intermediários. (Vale mencionar também que as redes "sem camada intermediária" — ou chamadas " perceptrons " — só conseguem aprender funções essencialmente lineares, mas, assim que houver pelo menos uma camada intermediária, é sempre possível, em princípio, aproximar qualquer função arbitrariamente bem, pelo menos se houver neurônios suficientes, embora, para tornar o treinamento viável, normalmente seja necessário algum tipo de regularização ou normalização .)

Certo, digamos que você já tenha definido uma determinada arquitetura de rede neural. Agora, surge a questão de obter dados para treinar a rede. Muitos dos desafios práticos relacionados a redes neurais — e aprendizado de máquina em geral — giram em torno da aquisição ou preparação dos dados de treinamento necessários. Em muitos casos (“aprendizado supervisionado”), deseja-se obter exemplos explícitos de entradas e das saídas esperadas. Assim, por exemplo, pode-se querer imagens etiquetadas com base em seu conteúdo ou algum outro atributo. E talvez seja necessário realizar a etiquetagem manualmente — geralmente com grande esforço. Mas, muitas vezes, é possível aproveitar algo que já foi feito ou usá-lo como uma espécie de proxy. Por exemplo, pode-se usar tags alt fornecidas para imagens na web. Ou, em um domínio diferente, pode-se usar legendas ocultas criadas para vídeos. Ou — para treinamento de tradução de idiomas — pode-se usar versões paralelas de páginas da web ou outros documentos que existem em diferentes idiomas.

tarefa específica? Novamente, é difícil estimar a partir de princípios básicos. Certamente, os requisitos podem ser drasticamente reduzidos usando "aprendizado por transferência" para "transferir" coisas como listas de características importantes que já foram aprendidas em outra rede. Mas, geralmente, as redes neurais precisam "ver muitos exemplos" para treinar bem. E, pelo menos para algumas tarefas, é um fato importante sobre redes neurais que os exemplos podem ser incrivelmente repetitivos. De fato, é uma estratégia padrão simplesmente mostrar a uma rede neural todos os exemplos disponíveis, repetidamente. Em cada uma dessas "rodadas de treinamento" (ou "épocas"), a rede neural estará em um estado pelo menos ligeiramente diferente e, de alguma forma, "lembrá-la" de um exemplo específico é útil para que ela "se lembre desse exemplo". (E, sim, talvez isso seja análogo à utilidade da repetição na memorização humana.)

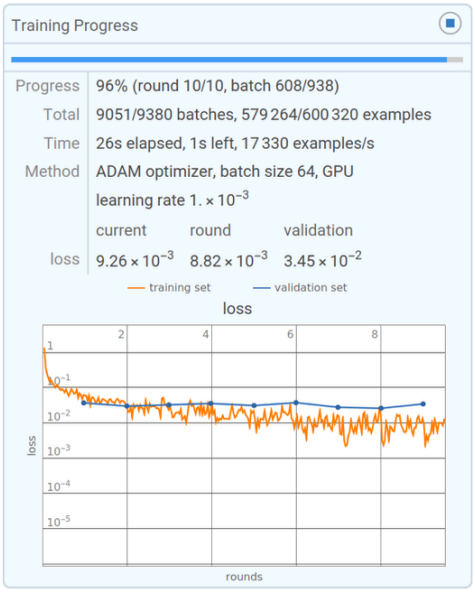
Mas, muitas vezes, repetir o mesmo exemplo várias vezes não é suficiente. Também é necessário mostrar as variações do exemplo na rede neural. E é um fato conhecido no campo das redes neurais que essas variações de "aumento de dados" não precisam ser sofisticadas para serem úteis. Modificar levemente as imagens com processamento básico pode torná-las essencialmente "como novas" para o treinamento da rede neural. Da mesma forma, quando se esgotam os vídeos reais, etc., para treinar carros autônomos, pode-se simplesmente obter dados executando simulações em um ambiente semelhante a um videogame, sem todos os detalhes de cenas reais.

Que tal algo como o ChatGPT? Bem, ele tem a ótima característica de poder realizar "aprendizado não supervisionado", o que facilita muito a obtenção de exemplos para treinamento. Lembre-se de que a tarefa básica do ChatGPT é descobrir como continuar um trecho de texto que lhe foi fornecido. Portanto, para obter "exemplos de treinamento", tudo o que é preciso fazer é pegar um trecho de texto, mascarar o final dele e usar isso como "entrada para treinamento" — com a "saída" sendo o trecho de texto completo, sem a máscara. Discutiremos isso mais adiante, mas o ponto principal é que — diferentemente, por exemplo, do aprendizado do conteúdo de imagens — não há necessidade de "etiquetagem explícita"; o ChatGPT pode, na prática, aprender diretamente com quaisquer exemplos de texto que lhe sejam fornecidos.

Certo, e quanto ao processo de aprendizagem em si em uma rede neural? No fim das contas, tudo se resume a determinar quais pesos melhor representam os exemplos de treinamento fornecidos. E existem diversas escolhas detalhadas e "configurações de hiperparâmetros" (chamadas assim porque os pesos podem ser considerados "parâmetros") que podem ser usadas para ajustar esse processo. Há diferentes opções de função de perda (soma dos quadrados, soma dos valores absolutos, etc.). Existem diferentes maneiras de minimizar a perda (o quanto avançar no espaço de pesos a cada passo, etc.). E há questões como o tamanho do "lote" de exemplos a ser apresentado

possível aplicar aprendizado de máquina (como fazemos, por exemplo, na Wolfram Language) para automatizar o aprendizado de máquina — e para definir automaticamente coisas como hiperparâmetros.

Mas, no fim das contas, todo o processo de treinamento pode ser caracterizado pela observação de como a perda diminui progressivamente (como neste monitor de progresso da Wolfram Language para um treinamento pequeno):



O que normalmente se observa é que a perda diminui por um tempo, mas eventualmente se estabiliza em um valor constante. Se esse valor for suficientemente pequeno, o treinamento pode ser considerado bem-sucedido; caso contrário, provavelmente é um sinal de que se deve tentar alterar a arquitetura da rede.

É possível determinar quanto tempo leva para a "curva de aprendizado" se estabilizar? Assim como em muitas outras coisas, parece haver relações de escala aproximadas de lei de potência que dependem do tamanho da rede neural e da quantidade de dados utilizada. Mas a conclusão geral é que treinar uma rede neural é difícil e exige muito esforço computacional. E, na prática, a grande maioria desse esforço é gasto realizando operações em matrizes de números, que é exatamente o que as GPUs fazem bem — razão pela qual o treinamento de redes neurais é normalmente limitado pela disponibilidade de GPUs.

No futuro, haverá maneiras fundamentalmente melhores de treinar redes neurais — ou, de forma geral, de fazer o que as redes neurais fazem? Quase certamente, eu diria. A ideia fundamental das redes neurais é criar uma "estrutura computacional" flexível a partir de um grande número de componentes simples (essencialmente idênticos) — e fazer com que essa "estrutura" possa ser modificada incrementalmente para aprender com exemplos. Nas redes neurais atuais, utiliza-se essencialmente os conceitos de cálculo — aplicados a números reais — para realizar essa modificação incremental. Mas

podem ser suficientes mesmo com os métodos atuais.

Com sistemas computacionais como os autômatos celulares , que basicamente operam em paralelo em muitos bits individuais, nunca ficou claro como realizar esse tipo de modificação incremental , mas não há razão para pensar que seja impossível. E, de fato, assim como ocorreu com o " avanço do aprendizado profundo em 2012 ", pode ser que essa modificação incremental seja efetivamente mais fácil em casos mais complexos do que em casos simples.

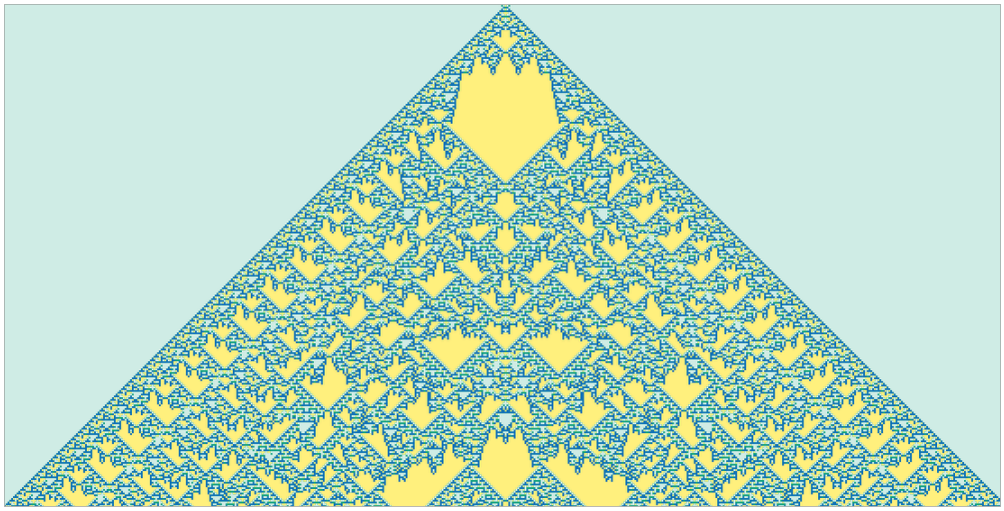
As redes neurais — talvez um pouco como os cérebros — são configuradas para terem uma rede essencialmente fixa de neurônios, sendo o que se modifica a força ("peso") das conexões entre eles. (Talvez, pelo menos em cérebros jovens, um número significativo de conexões totalmente novas também possa surgir.) Mas, embora essa possa ser uma configuração conveniente para a biologia, não está nada claro que seja sequer a melhor maneira de alcançar a funcionalidade de que precisamos. E algo que envolva o equivalente à reescrita progressiva da rede (talvez algo que lembre nosso Projeto de Física) pode muito bem ser, em última análise, melhor.

Mas mesmo dentro da estrutura das redes neurais existentes, há atualmente uma limitação crucial: o treinamento de redes neurais, como é feito hoje, é fundamentalmente sequencial, com os efeitos de cada lote de exemplos sendo propagados de volta para atualizar os pesos. E, de fato, com o hardware de computador atual — mesmo levando em consideração as GPUs — a maior parte de uma rede neural fica "ociosa" durante o treinamento, com apenas uma parte sendo atualizada por vez. E, em certo sentido, isso ocorre porque nossos computadores atuais tendem a ter memória separada de suas CPUs (ou GPUs). Mas, no cérebro, presume-se que seja diferente — com cada "elemento de memória" (ou seja, neurônio) sendo também um elemento computacional potencialmente ativo. E se pudéssemos configurar nosso hardware de computador futuro dessa maneira, talvez fosse possível realizar o treinamento de forma muito mais eficiente.

"Com certeza, uma rede grande o suficiente pode fazer qualquer coisa!"

As capacidades de algo como o ChatGPT parecem tão impressionantes que se poderia imaginar que, se pudéssemos simplesmente "continuar" e treinar redes neurais cada vez maiores, eventualmente seríamos capazes de "fazer tudo". E se estivermos falando de coisas que são facilmente acessíveis ao pensamento humano imediato, é bem possível que esse seja o caso. Mas a lição dos últimos séculos da ciência é que existem coisas que podem ser descobertas por processos formais, mas que não são facilmente acessíveis ao pensamento humano imediato.

computação. E, em última análise, a questão é o fenômeno da irredutibilidade computacional . Existem alguns cálculos que poderíamos pensar que exigiriam muitas etapas, mas que, na verdade, podem ser "reduzidos" a algo bastante imediato. Mas a descoberta da irredutibilidade computacional implica que isso nem sempre funciona. E, em vez disso, existem processos — provavelmente como o descrito abaixo — em que descobrir o que acontece inevitavelmente exige, essencialmente, rastrear cada etapa computacional:



Presume-se que os tipos de coisas que normalmente fazemos com o cérebro sejam escolhidos especificamente para evitar a irredutibilidade computacional. É preciso um esforço especial para fazer cálculos matemáticos mentalmente. E, na prática, é praticamente impossível "pensar" sobre as etapas de operação de qualquer programa não trivial apenas mentalmente.

Mas é claro que para isso temos computadores. E com computadores podemos facilmente realizar tarefas longas e computacionalmente irredutíveis. E o ponto crucial é que, em geral, não há atalhos para essas tarefas.

Sim, poderíamos memorizar muitos exemplos específicos do que acontece em um determinado sistema computacional. E talvez pudéssemos até identificar alguns padrões ("computacionalmente redutíveis") que nos permitiriam fazer algumas generalizações. Mas a questão é que a irredutibilidade computacional significa que nunca podemos garantir que o inesperado não aconteça — e é somente realizando o cálculo explicitamente que podemos determinar o que realmente ocorre em um caso específico.

E, no fim das contas, existe uma tensão fundamental entre a capacidade de aprendizado e a irredutibilidade computacional. O aprendizado envolve, na prática, comprimir dados aproveitando regularidades . Mas a irredutibilidade computacional implica que, em última análise, há um limite para quais regularidades podem existir.

— como autômatos celulares ou máquinas de Turing — em sistemas treináveis como redes neurais. E, de fato, tais dispositivos podem servir como boas "ferramentas" para a rede neural — assim como o Wolfram|Alpha pode ser uma boa ferramenta para o ChatGPT . Mas a irredutibilidade computacional implica que não se pode esperar "entrar" nesses dispositivos e fazê-los aprender.

Ou, dito de outra forma, existe uma relação de compromisso fundamental entre capacidade e treinabilidade: quanto mais se deseja que um sistema faça "uso real" de suas capacidades computacionais, mais ele demonstrará irredutibilidade computacional e menos será treinável. E quanto mais fundamentalmente treinável ele for, menos capaz será de realizar computação sofisticada.

(No caso do ChatGPT, em seu estado atual, a situação é ainda mais extrema, pois a rede neural usada para gerar cada token de saída é uma rede puramente "feed-forward", sem loops, e, portanto, não possui capacidade de realizar qualquer tipo de computação com "fluxo de controle" não trivial.)

É claro que alguém poderia questionar se é realmente importante ser capaz de realizar cálculos irredutíveis. E, de fato, durante grande parte da história da humanidade, isso não foi particularmente importante. Mas nosso mundo tecnológico moderno foi construído sobre a engenharia que utiliza, no mínimo, cálculos matemáticos — e, cada vez mais, cálculos mais gerais. E se observarmos o mundo natural, ele está repleto de cálculos irredutíveis — que estamos aos poucos aprendendo a emular e utilizar para nossos fins tecnológicos.

Sim, uma rede neural certamente pode perceber os tipos de regularidades no mundo natural que nós também poderíamos perceber facilmente com o "pensamento humano sem auxílio". Mas se quisermos resolver problemas que estão no âmbito da matemática ou da ciência da computação, a rede neural não será capaz de fazê-lo — a menos que ela efetivamente "use como ferramenta" um sistema computacional "comum".

Mas há algo potencialmente confuso em tudo isso. No passado, havia muitas tarefas — incluindo escrever redações — que presumíamos serem de alguma forma "fundamentalmente difíceis demais" para computadores. E agora que as vemos sendo realizadas por ferramentas como o ChatGPT, tendemos a pensar repentinamente que os computadores devem ter se tornado muito mais poderosos — em particular, superando coisas que eles já eram basicamente capazes de fazer (como computar progressivamente o comportamento de sistemas computacionais como autômatos celulares).

Mas essa não é a conclusão correta. Processos computacionalmente irredutíveis continuam sendo computacionalmente irredutíveis e continuam sendo fundamentalmente difíceis para computadores — mesmo que eles consigam calcular

— como escrever redações — que nós, humanos, conseguíamos fazer, mas que não imaginávamos que os computadores pudessem fazer, são, na verdade, de certa forma, computacionalmente mais fáceis do que pensávamos.

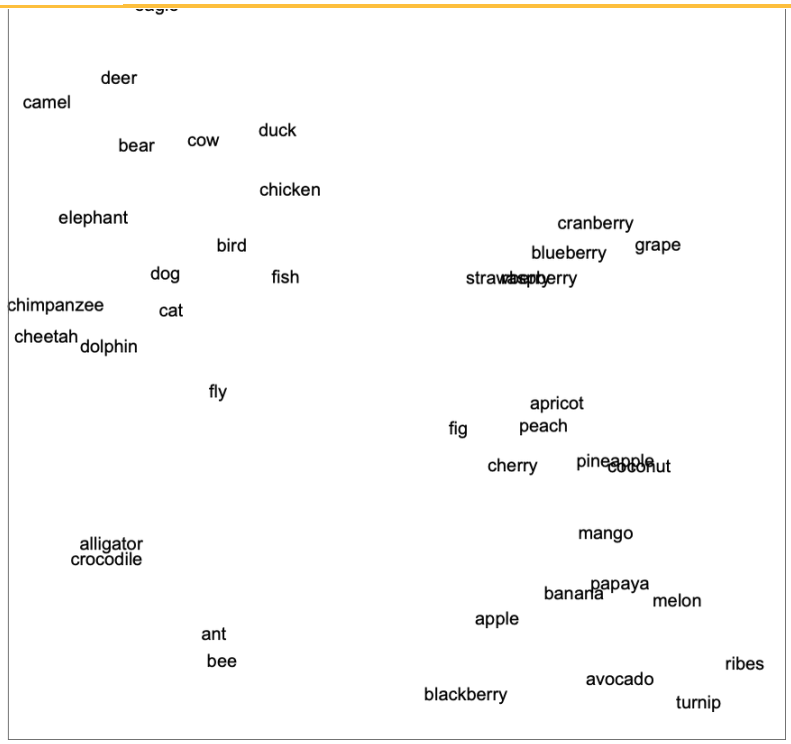
Em outras palavras, a razão pela qual uma rede neural pode ser bem-sucedida na escrita de uma redação é porque escrever uma redação acaba sendo um problema "computacionalmente mais superficial" do que pensávamos. E, de certa forma, isso nos aproxima de "ter uma teoria" sobre como nós, humanos, conseguimos realizar tarefas como escrever redações ou, de modo geral, lidar com a linguagem.

Se você tivesse uma rede neural suficientemente grande, sim, talvez conseguisse fazer tudo o que os humanos fazem com facilidade. Mas não conseguiria capturar o que o mundo natural em geral é capaz de fazer — ou o que as ferramentas que criamos a partir do mundo natural são capazes de fazer. E é o uso dessas ferramentas — tanto práticas quanto conceituais — que nos permitiu, nos últimos séculos, transcender os limites do que é acessível ao "pensamento humano puro e sem auxílio" e capturar, para fins humanos, mais do que existe no universo físico e computacional.

O conceito de incorporações

As redes neurais — pelo menos na sua configuração atual — são fundamentalmente baseadas em números. Portanto, se quisermos usá-las para trabalhar com algo como texto, precisaremos de uma maneira de representar nosso texto com números . E certamente poderíamos começar (essencialmente como o ChatGPT faz) simplesmente atribuindo um número a cada palavra do dicionário. Mas há uma ideia importante — que é central para o ChatGPT, por exemplo — que vai além disso. E é a ideia de "embeddings". Podemos pensar em um embedding como uma maneira de tentar representar a "essência" de algo por meio de uma matriz de números — com a propriedade de que "coisas próximas" são representadas por números próximos.

Assim, por exemplo, podemos pensar em um vetor de palavras como uma tentativa de organizar as palavras em uma espécie de "espaço de significado", no qual palavras que são de alguma forma "próximas em significado" aparecem próximas umas das outras no vetor. Os vetores reais que são usados — digamos, no ChatGPT — tendem a envolver grandes listas de números. Mas se projetarmos em 2D, podemos mostrar exemplos de como as palavras são organizadas pelo vetor:



E, sim, o que vemos captura de forma notável as impressões típicas do dia a dia. Mas como podemos construir um embedding desse tipo? A ideia geral é analisar grandes quantidades de texto (aqui, 5 bilhões de palavras da web) e observar o quão semelhantes são os "ambientes" em que diferentes palavras aparecem. Assim, por exemplo, "jacaré" e "crocodilo" frequentemente aparecem quase que indistintamente em frases semelhantes, o que significa que serão posicionados próximos uns dos outros no embedding. Já "nabo" e "águia" não costumam aparecer em frases semelhantes, então serão posicionados bem distantes um do outro no embedding.

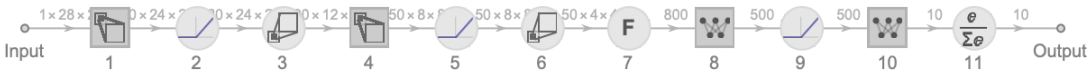
Mas como implementar algo assim usando redes neurais? Vamos começar falando sobre embeddings não para palavras, mas para imagens. Queremos encontrar uma maneira de caracterizar imagens por listas de números de forma que "imagens que consideramos semelhantes" recebam listas de números semelhantes.

Como podemos saber se devemos "considerar imagens semelhantes"? Bem, se nossas imagens forem, digamos, de dígitos manuscritos, podemos "considerar duas imagens semelhantes" se elas mostrarem o mesmo dígito. Anteriormente, discutimos uma rede neural treinada para reconhecer dígitos manuscritos. Podemos imaginar essa rede neural configurada de forma que, em sua saída final, ela classifique as imagens em 10 categorias diferentes, uma para cada dígito.

Mas e se "interceptássemos" o que está acontecendo dentro da rede neural antes da decisão final de que "é um '4'"? Poderíamos esperar que, dentro da rede neural, existissem números que caracterizassem as imagens como sendo "predominantemente semelhantes ao 4, mas um pouco semelhantes ao 2" ou algo do tipo. E a ideia é captar esses números para usá-los como elementos em um embedding.

imagem está próxima de qual outra imagem", consideramos uma tarefa bem definida (neste caso, reconhecimento de dígitos) para a qual podemos obter dados de treinamento explícitos — e então usamos o fato de que, ao realizar essa tarefa, a rede neural implicitamente precisa tomar o que equivale a "decisões de proximidade". Assim, em vez de termos que falar explicitamente sobre "proximidade de imagens", estamos apenas falando sobre a questão concreta de qual dígito uma imagem representa e, então, "deixamos para a rede neural" determinar implicitamente o que isso implica sobre a "proximidade de imagens".

Então, como isso funciona em detalhes para a rede de reconhecimento de dígitos? Podemos pensar na rede como sendo composta por 11 camadas sucessivas, que podemos resumir iconicamente assim (com as funções de ativação mostradas como camadas separadas):



No início, alimentamos a primeira camada com imagens reais, representadas por matrizes 2D de valores de pixels. E no final — da última camada — obtemos uma matriz de 10 valores, que podemos interpretar como a certeza da rede de que a imagem corresponde a cada um dos dígitos de 0 a 9.

Insira a imagem  e os valores dos neurônios nessa última camada serão:

$$\{1.42071\times10^{-22},\ 7.69857\times10^{-14},\ 1.9653\times10^{-16},\ 5.55229\times10^{-21},\ 1.,$$
$$8.33841\times10^{-14},\ 6.89742\times10^{-17},\ 6.52282\times10^{-19},\ 6.51465\times10^{-12},\ 1.97509\times10^{-14}\}$$

Em outras palavras, a essa altura a rede neural está "incrivelmente certa" de que essa imagem é um 4 — e para obter a saída "4", basta selecionar a posição do neurônio com o maior valor.

Mas e se olharmos um passo antes? A última operação na rede é uma chamada softmax , que tenta "forçar a certeza". Mas antes de ser aplicada, os valores dos neurônios são:

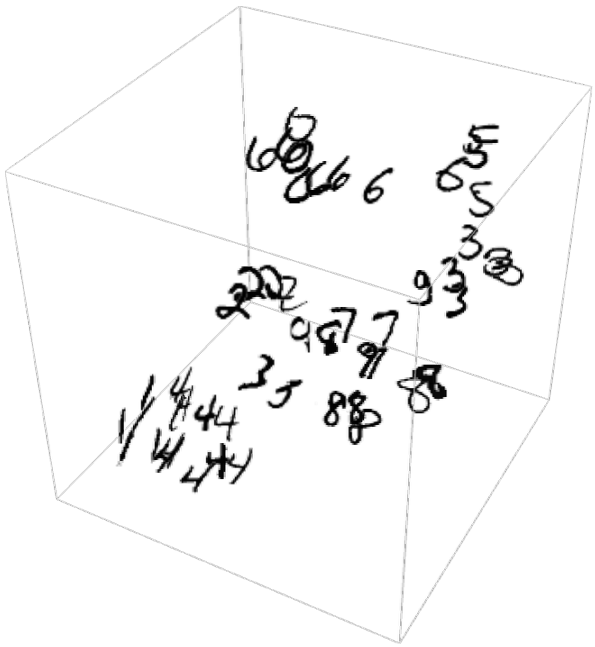
$$\{-26.134,\ -6.02347,\ -11.994,\ -22.4684,$$
$$24.1717,\ -5.94363,\ -13.0411,\ -17.7021,\ -1.58528,\ -7.38389\}$$

O neurônio que representa o número “4” ainda possui o maior valor numérico. Mas também há informações nos valores dos outros neurônios. E podemos esperar que essa lista de números possa, de certa forma, ser usada para caracterizar a “essência” da imagem — e, portanto, fornecer algo que possamos usar como um vetor de incorporação. Assim, por exemplo, cada um dos números 4 aqui possui uma “assinatura” (ou “vetor de incorporação de características”) ligeiramente diferente — todas muito diferentes dos números 8:



Aqui, estamos essencialmente usando 10 números para caracterizar nossas imagens. Mas, muitas vezes, é melhor usar muito mais do que isso. Por exemplo, em nossa rede de reconhecimento de dígitos, podemos obter uma matriz de 500 números acessando a camada anterior. E essa provavelmente é uma matriz razoável para usar como uma "incorporação de imagem".

Se quisermos criar uma visualização explícita do "espaço da imagem" para dígitos manuscritos, precisamos "reduzir a dimensão", projetando o vetor de 500 dimensões que obtivemos, digamos, em um espaço 3D:



Acabamos de falar sobre a criação de uma caracterização (e, portanto, de um embedding) para imagens, baseada na identificação da similaridade entre elas, determinando se (de acordo com nosso conjunto de treinamento) correspondem ao mesmo dígito manuscrito. Podemos fazer o mesmo de forma muito mais geral para imagens se tivermos um conjunto de treinamento que identifique, por exemplo, qual dos 5.000 tipos comuns de objetos (gato, cachorro, cadeira, etc.) cada imagem representa. Dessa forma, podemos criar um embedding de imagem "ancorado" pela nossa identificação de objetos comuns, mas que então "generaliza em torno disso" de acordo com o comportamento da rede neural. A questão é que, na medida em que esse comportamento se alinha com a forma como nós, humanos, percebemos e interpretamos imagens, esse embedding acabará sendo algo que "nos parece correto" e será útil na prática para realizar tarefas que exigem julgamento humano.

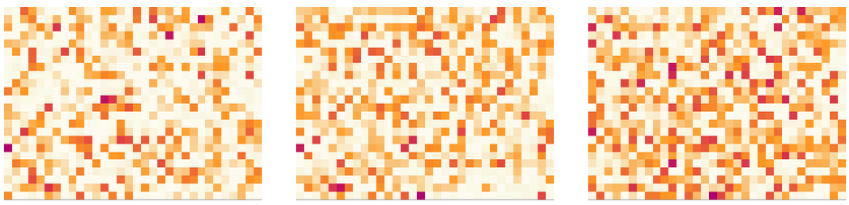
Certo, então como seguimos a mesma abordagem para encontrar representações vetoriais de palavras? A chave é começar com uma tarefa sobre palavras para a qual

palavras". Imagine que nos seja dado "o gato ____". Com base em um grande conjunto de textos (digamos, o conteúdo textual da web), quais são as probabilidades de diferentes palavras que podem "preencher a lacuna"? Ou, alternativamente, dado "____ preto ____", quais são as probabilidades de diferentes "palavras complementares"?

Como configuramos esse problema para uma rede neural? Em última análise, precisamos formular tudo em termos de números. E uma maneira de fazer isso é simplesmente atribuir um número único a cada uma das cerca de 50.000 palavras comuns em inglês. Por exemplo, "the" (o/a) poderia ser 914 e "cat" (gato/a) (com um espaço antes) poderia ser 3542. (E esses são os números reais usados pelo GPT-2.) Então, para o problema "the ____ cat" (o/a ____ gato/a), nossa entrada poderia ser {914, 3542}. Como deveria ser a saída? Bem, deveria ser uma lista de cerca de 50.000 números que efetivamente fornecem as probabilidades para cada uma das possíveis palavras de preenchimento. E, mais uma vez, para encontrar um embedding, queremos "interceptar" o "interior" da rede neural pouco antes de ela "chegar à sua conclusão" — e então coletar a lista de números que ocorre ali, e que podemos considerar como "caracterizando cada palavra".

Certo, então como são essas caracterizações? Nos últimos 10 anos, houve uma série de sistemas diferentes desenvolvidos (word2vec , GloVe , BERT , GPT , ...), cada um baseado em uma abordagem diferente de rede neural. Mas, em última análise, todos eles pegam palavras e as caracterizam por listas de centenas a milhares de números.

Em sua forma bruta, esses “vetores de incorporação” são bastante desprovidos de informação. Por exemplo, veja o que o GPT-2 produz como vetores de incorporação brutos para três palavras específicas:



cat dog chair

Se fizermos coisas como medir as distâncias entre esses vetores, podemos encontrar coisas como "proximidades" entre palavras. Mais tarde, discutiremos em mais detalhes o que poderíamos considerar o significado "cognitivo" de tais representações vetoriais. Mas, por ora, o ponto principal é que temos uma maneira útil de transformar palavras em coleções de números "amigáveis a redes neurais".

Mas, na verdade, podemos ir além de simplesmente caracterizar palavras por conjuntos de números; também podemos fazer isso para sequências de palavras ou, de fato, blocos inteiros de texto. E é assim que o ChatGPT lida com as coisas. Ele pega o texto que tem até o momento e gera um vetor de incorporação para representá-lo. Então, seu

seguida. E ele representa sua resposta como uma lista de números que essencialmente fornecem as probabilidades para cada uma das cerca de 50.000 palavras possíveis.

(Tecnicamente, o ChatGPT não lida com palavras, mas sim com "tokens" — unidades linguísticas convenientes que podem ser palavras inteiras ou apenas partes como "pre", "ing" ou "ized". Trabalhar com tokens facilita para o ChatGPT lidar com palavras raras, compostas e não inglesas e, às vezes, para o bem ou para o mal, inventar novas palavras.)

Dentro do ChatGPT

Certo, então finalmente estamos prontos para discutir o que há dentro do ChatGPT. E sim, em última análise, trata-se de uma gigantesca rede neural — atualmente uma versão da chamada rede GPT-3 com 175 bilhões de pesos. Em muitos aspectos, esta é uma rede neural muito semelhante às outras que discutimos. Mas é uma rede neural especialmente projetada para lidar com a linguagem. E sua característica mais notável é um elemento da arquitetura de redes neurais chamado "transformador".

Nas primeiras redes neurais que discutimos acima, cada neurônio em qualquer camada era basicamente conectado (pelo menos com algum peso) a todos os neurônios da camada anterior. Mas esse tipo de rede totalmente conectada é (presumivelmente) um exagero se estivermos trabalhando com dados que possuem uma estrutura particular e conhecida. Assim, por exemplo, nos estágios iniciais do processamento de imagens, é típico usar as chamadas redes neurais convolucionais (“convnets”), nas quais os neurônios são dispostos em uma grade análoga aos pixels da imagem — e conectados apenas a neurônios próximos na grade.

A ideia dos Transformers é fazer algo pelo menos um pouco semelhante para sequências de tokens que compõem um texto. Mas, em vez de simplesmente definir uma região fixa na sequência sobre a qual podem existir conexões, os Transformers introduzem a noção de " atenção " — e a ideia de "prestar atenção" mais a algumas partes da sequência do que a outras. Talvez um dia faça sentido simplesmente começar com uma rede neural genérica e fazer toda a personalização por meio do treinamento. Mas, pelo menos por enquanto, parece ser crucial na prática "modularizar" as coisas — como os Transformers fazem, e provavelmente como nossos cérebros também fazem.

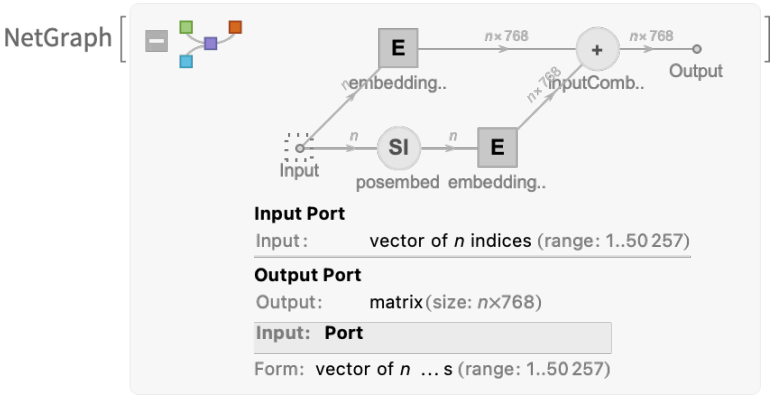
Certo, então o que o ChatGPT (ou melhor, a rede GPT-3 na qual ele se baseia) realmente faz? Lembre-se de que seu objetivo geral é continuar o texto de uma maneira "razoável", com base no que aprendeu durante o treinamento (que consiste em analisar bilhões de páginas de texto da web, etc.). Portanto, a qualquer momento, ele tem uma certa quantidade de texto — e seu objetivo é encontrar uma opção apropriada para o próximo token a ser adicionado.

corresponde ao texto até o momento e encontra um vetor de incorporação (ou seja, uma matriz de números) que os representa. Em seguida, opera sobre esse vetor de incorporação — de uma maneira "padrão de rede neural", com os valores "se propagando" pelas camadas sucessivas da rede — para produzir um novo vetor de incorporação (ou seja, uma nova matriz de números). Depois, pega a última parte dessa matriz e gera a partir dela uma matriz com cerca de 50.000 valores que se transformam em probabilidades para diferentes possíveis próximos tokens. (E, sim, acontece que há aproximadamente o mesmo número de tokens usados que de palavras comuns em inglês, embora apenas cerca de 3.000 dos tokens sejam palavras inteiras, e o restante sejam fragmentos.)

Um ponto crucial é que cada parte desse processo é implementada por uma rede neural, cujos pesos são determinados pelo treinamento de ponta a ponta da rede. Em outras palavras, na prática, nada além da arquitetura geral é "explicitamente projetado"; tudo é simplesmente "aprendido" a partir dos dados de treinamento.

Existem, no entanto, muitos detalhes na forma como a arquitetura é configurada — refletindo diversos tipos de experiência e conhecimento sobre redes neurais. E — embora isso certamente seja entrar em detalhes muito específicos — acho útil falar sobre alguns desses detalhes, principalmente para ter uma ideia do que envolve a construção de algo como o ChatGPT.

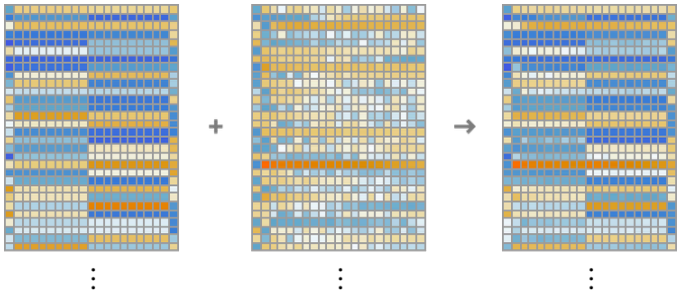
Primeiro vem o módulo de incorporação. Aqui está uma representação esquemática em Wolfram Language para ele no GPT-2:



A entrada é um vetor de *n* tokens (representados, como na seção anterior, por inteiros de 1 a aproximadamente 50.000). Cada um desses tokens é convertido (por uma rede neural de camada única) em um vetor de incorporação (de comprimento 768 para o GPT-2 e 12.288 para o GPT-3 do ChatGPT). Enquanto isso, existe um "caminho secundário" que recebe a sequência de posições (inteiras) dos tokens e, a partir desses inteiros, cria outro vetor de incorporação. Finalmente, os vetores de incorporação do valor do token e da posição do token são somados — para produzir a sequência final de vetores de incorporação do módulo de incorporação.

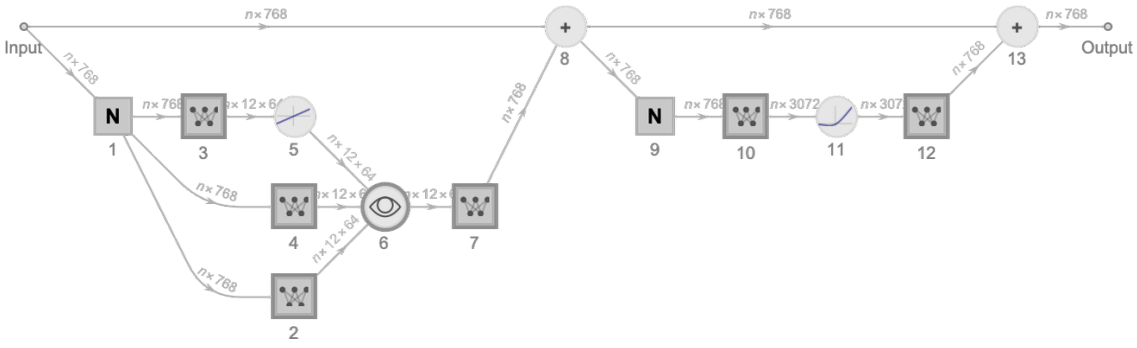
Não creio que haja uma explicação científica específica para isso. É apenas uma tentativa de várias abordagens diferentes, e esta parece funcionar. E faz parte da tradição das redes neurais que — de certa forma — contanto que a configuração inicial esteja "aproximadamente correta", geralmente é possível refinar os detalhes apenas com treinamento suficiente, sem precisar realmente "entender em um nível técnico" como a rede neural acabou se configurando.

Eis o que o módulo de incorporação faz, operando na string *hello hello hello hello hello hello hello hello hello bye* ...



Os elementos do vetor de incorporação para cada token são mostrados na parte inferior da página, e na horizontal vemos primeiro uma sequência de incorporações de " *olá ", seguida por uma sequência de incorporações de " tchau* ". O segundo array acima é a incorporação posicional — cuja estrutura aparentemente aleatória é simplesmente o que "acabou sendo aprendido" (neste caso, no GPT-2).

Certo, então, após o módulo de incorporação, vem o "evento principal" do Transformer: uma sequência dos chamados "blocos de atenção" (12 para o GPT-2, 96 para o GPT-3 do ChatGPT). É tudo bastante complexo — e lembra sistemas de engenharia grandes e difíceis de entender, ou, aliás, sistemas biológicos. Mas enfim, aqui está uma representação esquemática de um único "bloco de atenção" (para o GPT-2):

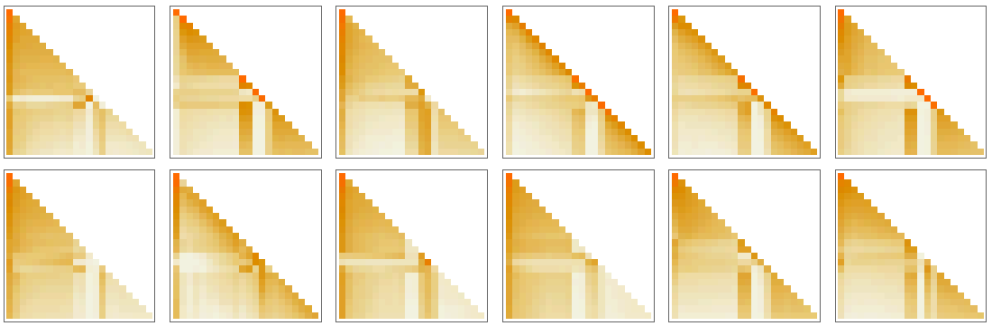


Dentro de cada bloco de atenção, há uma coleção de "cabeças de atenção" (12 para o GPT-2, 96 para o GPT-3 do ChatGPT) — cada uma operando independentemente em diferentes blocos de valores no vetor de incorporação. (E, sim, não sabemos nenhum motivo específico pelo qual seja uma boa ideia dividir o vetor de incorporação, ou o que

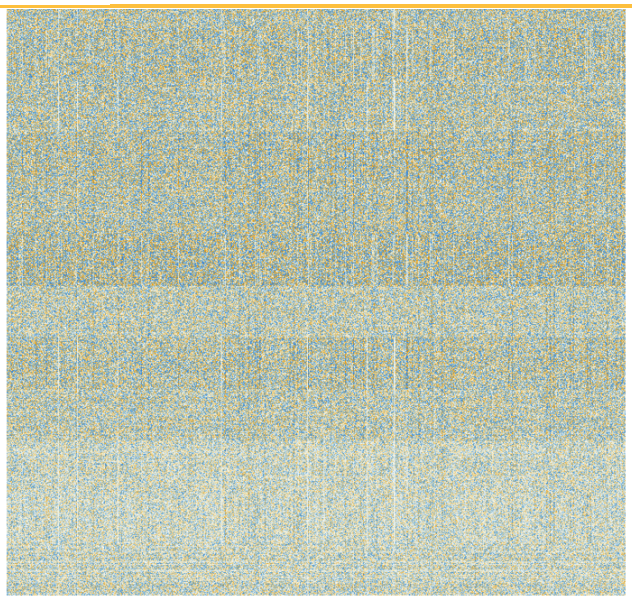
"funcionam".)

Certo, então o que fazem os mecanismos de atenção? Basicamente, eles são uma forma de "revisar" a sequência de tokens (ou seja, o texto produzido até o momento) e "compactar o passado" de uma forma útil para encontrar o próximo token. Na primeira seção acima, falamos sobre o uso de probabilidades de 2-gramas para selecionar palavras com base em seus predecessores imediatos. O que o mecanismo de "atenção" nos Transformers faz é permitir que a "atenção" seja dada até mesmo a palavras muito anteriores — capturando, assim, a maneira como, por exemplo, verbos podem se referir a substantivos que aparecem muitas palavras antes deles em uma frase.

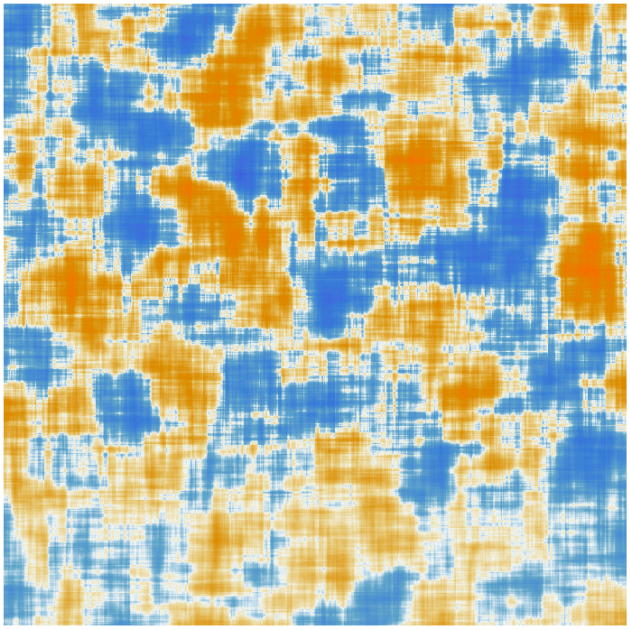
Em um nível mais detalhado, o que uma cabeça de atenção faz é recombinar partes dos vetores de incorporação associados a diferentes tokens, com certos pesos. Assim, por exemplo, as 12 cabeças de atenção no primeiro bloco de atenção (no GPT-2) têm os seguintes padrões de "pesos de recombinação" ("olhar para trás até o início da sequência de tokens") para a string " *hello , bye " acima:*



Após ser processado pelos núcleos de atenção, o “vetor de incorporação reponderado” resultante (de comprimento 768 para o GPT-2 e 12.288 para o GPT-3 do ChatGPT) é passado por uma camada de rede neural “totalmente conectada” padrão . É difícil entender exatamente o que essa camada faz. Mas aqui está um gráfico da matriz de pesos 768×768 que ela utiliza (aqui para o GPT-2):



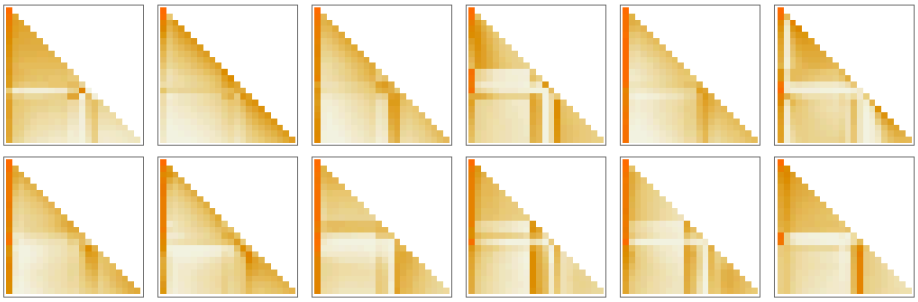
Ao calcular médias móveis de 64×64, alguma estrutura (semelhante a um passeio aleatório) começa a surgir:



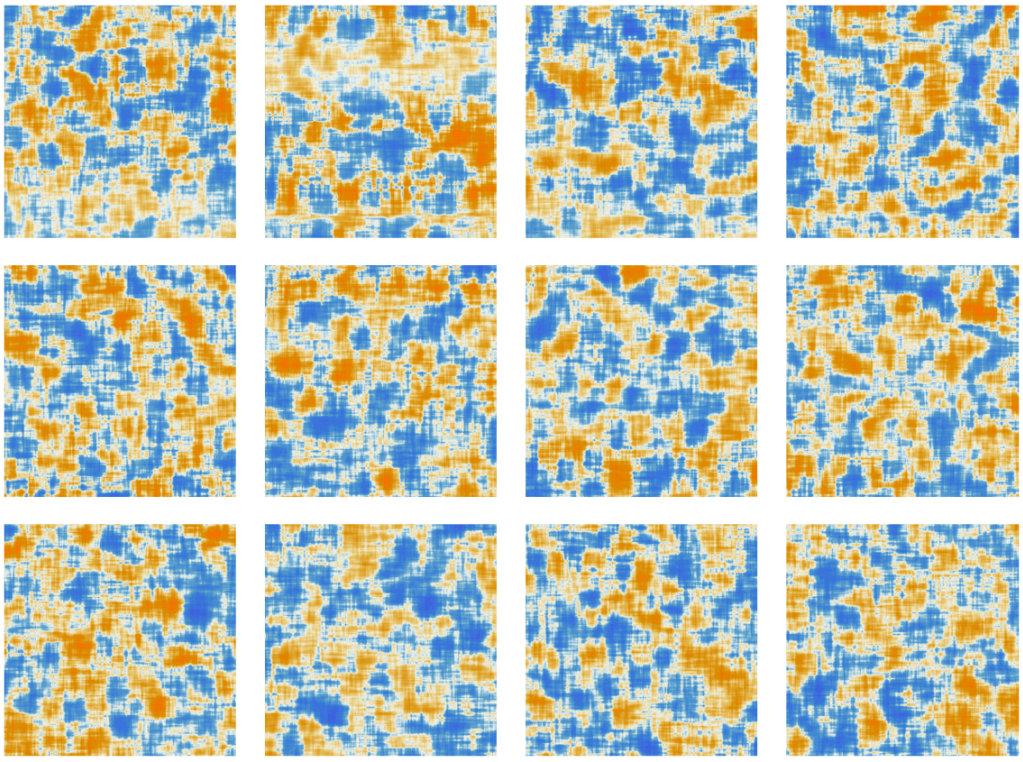
O que determina essa estrutura? Em última análise, presume-se que seja alguma "codificação de rede neural" de características da linguagem humana. Mas, até o momento, quais seriam essas características é um mistério. Na prática, estamos "abrindo o cérebro do ChatGPT" (ou pelo menos do GPT-2) e descobrindo que, sim, é complexo por dentro e que não o compreendemos completamente — embora, no final, ele produza linguagem humana reconhecível.

Certo, então, após passar por um bloco de atenção, obtemos um novo vetor de incorporação — que é então sucessivamente passado por blocos de atenção adicionais (um total de 12 para o GPT-2; 96 para o GPT-3). Cada bloco de atenção tem seu próprio padrão específico de pesos de "atenção" e "totalmente conectados". Aqui está a

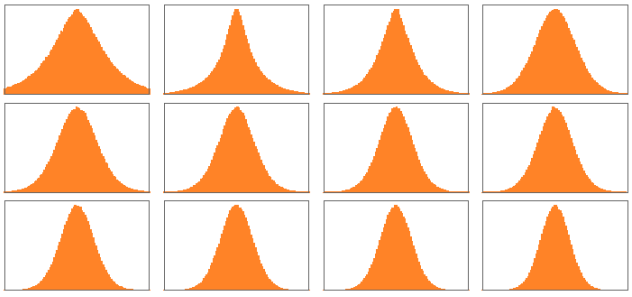
bloco de atenção:



E aqui estão as “matrizes” (com média móvel) para as camadas totalmente conectadas:



Curiosamente, embora essas “matrizes de pesos” em diferentes blocos de atenção pareçam bastante semelhantes, as distribuições dos tamanhos dos pesos podem ser um pouco diferentes (e nem sempre são gaussianas):



Então, depois de passar por todos esses blocos de atenção, qual é o efeito final do Transformer? Essencialmente, ele transforma a coleção original de embeddings da sequência de tokens em uma coleção final. E a maneira específica como o ChatGPT

produzir uma lista de probabilidades de qual token deve vir a seguir.

Em resumo, esse é o funcionamento interno do ChatGPT. Pode parecer complicado (principalmente devido às suas muitas "escolhas de engenharia" inevitavelmente um tanto arbitrárias), mas, na verdade, os elementos fundamentais envolvidos são surpreendentemente simples. Afinal, estamos lidando apenas com uma rede neural composta de "neurônios artificiais", cada um realizando a simples operação de receber um conjunto de entradas numéricas e combiná-las com determinados pesos.

A entrada original do ChatGPT é uma matriz de números (os vetores de incorporação dos tokens até o momento), e o que acontece quando o ChatGPT "executa" para produzir um novo token é que esses números simplesmente "se propagam" pelas camadas da rede neural, com cada neurônio "fazendo sua função" e passando o resultado para os neurônios da camada seguinte. Não há loops ou "retrocessos". Tudo simplesmente "se propaga" pela rede.

Trata-se de uma configuração muito diferente de um sistema computacional típico — como uma máquina de Turing — no qual os resultados são repetidamente "reprocessados" pelos mesmos elementos computacionais. Aqui — pelo menos na geração de um determinado resultado — cada elemento computacional (ou seja, neurônio) é usado apenas uma vez.

Mas, em certo sentido, ainda existe um "loop externo" que reutiliza elementos computacionais mesmo no ChatGPT. Isso porque, quando o ChatGPT vai gerar um novo token, ele sempre "lê" (ou seja, recebe como entrada) toda a sequência de tokens que o precedem, incluindo tokens que o próprio ChatGPT "escreveu" anteriormente. E podemos pensar nessa configuração como significando que o ChatGPT, pelo menos em seu nível mais externo, envolve um "loop de feedback", embora cada iteração seja explicitamente visível como um token que aparece no texto que ele gera.

Mas voltemos ao cerne do ChatGPT: a rede neural que é usada repetidamente para gerar cada token. Em certo nível, é muito simples: uma coleção inteira de neurônios artificiais idênticos. Algumas partes da rede consistem em camadas de neurônios (" totalmente conectadas "), nas quais cada neurônio em uma determinada camada está conectado (com algum peso) a todos os neurônios da camada anterior. Mas, particularmente com sua arquitetura Transformer, o ChatGPT possui partes com mais estrutura, nas quais apenas neurônios específicos em diferentes camadas estão conectados. (Claro, ainda se poderia dizer que "todos os neurônios estão conectados" — mas alguns simplesmente têm peso zero.)

Além disso, existem aspectos da rede neural no ChatGPT que não são naturalmente concebidos como sendo compostos apenas por camadas “homogêneas”. Por exemplo — como indica o resumo acima — dentro de um bloco de atenção, existem locais onde

de processamento" diferente, potencialmente envolvendo um número diferente de camadas, e recombinando-se somente mais tarde. Mas, embora essa possa ser uma representação conveniente do que está acontecendo, é sempre possível, pelo menos em princípio, pensar em "preencher densamente" as camadas, mas com alguns pesos iguais a zero.

Se analisarmos o caminho mais longo no ChatGPT, veremos que ele envolve cerca de 400 camadas (principais) — um número que, de certa forma, não é tão grande. Mas existem milhões de neurônios — com um total de 175 bilhões de conexões e, portanto, 175 bilhões de pesos. E é importante notar que, cada vez que o ChatGPT gera um novo token, ele precisa realizar um cálculo envolvendo cada um desses pesos. Do ponto de vista da implementação, esses cálculos podem ser organizados "por camada" em operações de matriz altamente paralelas, que podem ser executadas convenientemente em GPUs. Mas, para cada token produzido, ainda são necessários 175 bilhões de cálculos (e, no final, um pouco mais) — então, sim, não é surpreendente que a geração de um texto longo com o ChatGPT possa levar um tempo considerável.

Mas, no fim das contas, o mais notável é que todas essas operações — individualmente tão simples quanto são — conseguem, de alguma forma, juntas, gerar um texto tão "semelhante ao humano". É preciso enfatizar novamente que (pelo menos até onde sabemos) não existe uma "razão teórica fundamental" para que algo assim funcione. E, na verdade, como discutiremos, acho que devemos encarar isso como uma descoberta científica — potencialmente surpreendente —: que, de alguma forma, em uma rede neural como a do ChatGPT, é possível capturar a essência do que o cérebro humano consegue fazer na geração da linguagem.

O treinamento do ChatGPT

Certo, então já apresentamos um resumo de como o ChatGPT funciona depois de configurado. Mas como ele foi configurado? Como foram determinados todos esses 175 bilhões de pesos em sua rede neural? Basicamente, eles são o resultado de um treinamento em larga escala, baseado em um enorme conjunto de textos — na web, em livros, etc. — escritos por humanos. Como dissemos, mesmo com todos esses dados de treinamento, certamente não é óbvio que uma rede neural seja capaz de produzir com sucesso um texto "semelhante ao humano". E, mais uma vez, parece haver detalhes de engenharia necessários para que isso aconteça. Mas a grande surpresa — e descoberta — do ChatGPT é que isso seja possível. E que — na prática — uma rede neural com "apenas" 175 bilhões de pesos possa criar um "modelo razoável" do texto escrito por humanos.

Nos tempos modernos, existe uma enorme quantidade de textos escritos por humanos disponíveis em formato digital. A internet pública possui pelo menos vários bilhões de

incluirmos páginas da web privadas, esse número pode ser pelo menos 100 vezes maior. Até o momento, mais de 5 milhões de livros digitalizados foram disponibilizados (de um total de cerca de 100 milhões já publicados), o que representa mais 100 bilhões de palavras. E isso sem mencionar o texto derivado da fala em vídeos, etc. (A título de comparação pessoal, minha produção total de material publicado ao longo da vida foi um pouco inferior a 3 milhões de palavras, e nos últimos 30 anos escrevi cerca de 15 milhões de palavras em e-mails e digitei, no total, talvez 50 milhões de palavras — e somente nos últimos dois anos, falei mais de 10 milhões de palavras em transmissões ao vivo . E sim, eu treinaria um bot com tudo isso.)

Mas, tudo bem, com todos esses dados, como treinar uma rede neural? O processo básico é muito parecido com o que discutimos nos exemplos simples acima. Você apresenta um conjunto de exemplos e, em seguida, ajusta os pesos da rede para minimizar o erro ("perda") que a rede comete nesses exemplos. O principal fator que torna a "retropropagação" do erro custosa é que, a cada iteração, todos os pesos da rede geralmente mudam, mesmo que minimamente, e há muitos pesos para lidar. (O cálculo reverso em si geralmente é apenas um pouco mais complexo do que o cálculo direto.)

Com o hardware de GPU moderno, é simples calcular os resultados de lotes de milhares de exemplos em paralelo. Mas, quando se trata de atualizar os pesos na rede neural, os métodos atuais exigem que isso seja feito basicamente lote por lote. (E, sim, é provavelmente aqui que os cérebros reais — com seus elementos combinados de computação e memória — têm, pelo menos por enquanto, uma vantagem arquitetônica.)

Mesmo nos casos aparentemente simples de aprendizado de funções numéricas que discutimos anteriormente, descobrimos que muitas vezes precisávamos de milhões de exemplos para treinar uma rede com sucesso, pelo menos partindo do zero. Então, quantos exemplos isso significa que precisaremos para treinar um modelo de linguagem "semelhante à humana"? Não parece haver uma maneira "teórica" fundamental de saber. Mas, na prática, o ChatGPT foi treinado com sucesso em algumas centenas de bilhões de palavras de texto.

Parte do texto foi apresentado várias vezes, outra parte apenas uma vez. De alguma forma, porém, o sistema "captou o que precisava" a partir do texto visualizado. Mas, dado esse volume de texto para aprendizado, qual o tamanho ideal da rede neural para "aprender bem"? Novamente, ainda não temos uma resposta teórica fundamental. Em última análise — como discutiremos mais adiante — presume-se que exista um certo "conteúdo algorítmico total" na linguagem humana e no que os humanos normalmente expressam com ela. Mas a próxima questão é qual a eficiência de uma rede neural na

não sabemos — embora o sucesso do ChatGPT sugira que seja razoavelmente eficiente.

E, no fim das contas, podemos simplesmente observar que o ChatGPT realiza seu trabalho utilizando algumas centenas de bilhões de pesos — um número comparável ao total de palavras (ou tokens) dos dados de treinamento fornecidos. De certa forma, é talvez surpreendente (embora observado empiricamente também em análogos menores do ChatGPT) que o "tamanho da rede" que parece funcionar bem seja tão comparável ao "tamanho dos dados de treinamento". Afinal, certamente não é que "dentro do ChatGPT" todo aquele texto da web, livros e assim por diante esteja "armazenado diretamente". Porque o que realmente está dentro do ChatGPT é um conjunto de números — com um pouco menos de 10 dígitos de precisão — que representam algum tipo de codificação distribuída da estrutura agregada de todo aquele texto.

Em outras palavras, podemos nos perguntar qual é o "conteúdo informativo efetivo" da linguagem humana e o que normalmente é dito com ela. Existe o corpus bruto de exemplos de linguagem. E existe a representação na rede neural do ChatGPT. Essa representação provavelmente está longe da representação "algoritmicamente mínima" (como discutiremos abaixo). Mas é uma representação facilmente utilizável pela rede neural. E nessa representação parece haver, no final, pouca "compressão" dos dados de treinamento; em média, parece que basta um pouco menos de um peso da rede neural para carregar o "conteúdo informativo" de uma palavra dos dados de treinamento.

Quando executamos o ChatGPT para gerar texto, basicamente precisamos usar cada peso uma única vez. Portanto, se houver n pesos, teremos cerca de n etapas computacionais para realizar — embora, na prática, muitas delas possam ser executadas em paralelo em GPUs. Mas se precisarmos de cerca de n palavras de dados de treinamento para configurar esses pesos, então, com base no que dissemos acima, podemos concluir que precisaremos de cerca de n^2 *etapas* computacionais para treinar a rede — e é por isso que, com os métodos atuais, acabamos falando em custos de treinamento na casa dos bilhões de dólares.

Além do treinamento básico

A maior parte do esforço no treinamento do ChatGPT é gasto "mostrando a ele" grandes quantidades de texto existente da web, livros, etc. Mas acontece que há outra parte — aparentemente bastante importante — também.

Assim que termina seu "treinamento bruto" com o conjunto original de textos que lhe foi apresentado, a rede neural do ChatGPT está pronta para começar a gerar seu próprio texto, a partir de prompts, etc. Mas, embora os resultados possam parecer razoáveis em muitos casos, eles tendem — principalmente em textos mais longos — a "desviar-se" de maneiras que muitas vezes não condizem com a intuição humana. Não é

tradicionais do texto. Mas é algo que leitores humanos percebem com facilidade.

Uma ideia fundamental na construção do ChatGPT foi dar um passo além da "leitura passiva" de conteúdos como a web: permitir que humanos interagissem ativamente com o ChatGPT, observassem o que ele produzia e, na prática, fornecessem feedback sobre "como ser um bom chatbot". Mas como a rede neural pode usar esse feedback? O primeiro passo é simplesmente pedir que humanos avaliem os resultados da rede neural. Em seguida, um novo modelo de rede neural é construído para tentar prever essas avaliações. Agora, esse modelo de previsão pode ser executado — essencialmente como uma função de perda — na rede original, permitindo que ela seja "aprimorada" pelo feedback humano recebido. E os resultados na prática parecem ter um grande impacto no sucesso do sistema em produzir resultados "semelhantes aos humanos".

Em geral, é interessante notar como a rede "originalmente treinada" parece precisar de tão pouco "alteração" para seguir direções específicas de forma útil. Poderíamos imaginar que, para a rede se comportar como se tivesse "aprendido algo novo", seria necessário executar um algoritmo de treinamento, ajustar os pesos e assim por diante.

Mas não é bem assim. Na verdade, parece ser suficiente dizer ao ChatGPT uma única vez — como parte do comando fornecido — e ele consegue usar essa informação com sucesso ao gerar o texto. E, mais uma vez, o fato de isso funcionar é, a meu ver, uma pista importante para entender o que o ChatGPT "realmente faz" e como isso se relaciona com a estrutura da linguagem e do pensamento humanos.

Há certamente algo bastante humano nisso: pelo menos depois de todo o pré-treinamento, você pode dizer algo a ela apenas uma vez e ela consegue "lembrar" — pelo menos "por tempo suficiente" para gerar um texto usando essa informação. Então, o que acontece em um caso como esse? Pode ser que "tudo o que você possa dizer a ela já esteja lá em algum lugar" — e você esteja apenas a guiando para o lugar certo. Mas isso não parece plausível. Em vez disso, o que parece mais provável é que, sim, os elementos já estejam lá, mas os detalhes sejam definidos por algo como uma "trajetória entre esses elementos", e é isso que você está introduzindo quando diz algo a ela.

E, de fato, assim como acontece com os humanos, se você disser a uma criatura algo bizarro e inesperado que não se encaixa de forma alguma na estrutura que ela conhece, parece que ela não conseguirá "integrar" isso com sucesso. Ela só consegue "integrar" se estiver basicamente se apoiando, de uma maneira relativamente simples, na estrutura que já possui.

Vale ressaltar novamente que existem inevitavelmente "limitações algorítmicas" para o que a rede neural consegue "captar". Dê a ela regras "superficiais" do tipo "isto leva àquilo", etc., e a rede neural provavelmente conseguirá representá-las e reproduzi-las perfeitamente — e, de fato, o que ela "já sabe" da linguagem lhe dará um padrão

envolva muitas etapas potencialmente irredutíveis do ponto de vista computacional, e ela simplesmente não funcionará. (Lembre-se de que, a cada etapa, ela está apenas "alimentando dados adiante" em sua rede, nunca entrando em loop, exceto para gerar novos tokens.)

É claro que a rede pode aprender a resposta para cálculos específicos "irredutíveis". Mas, assim que houver um número combinatório de possibilidades, nenhuma abordagem do tipo "consulta em tabela" funcionará. E então, sim, assim como os humanos, chegou a hora de as redes neurais "alcançarem" e utilizarem ferramentas computacionais reais. (E sim, o Wolfram|Alpha e o Wolfram Language são particularmente adequados , porque foram criados para "conversar sobre coisas do mundo", assim como as redes neurais de modelo de linguagem.)

O que realmente permite que o ChatGPT funcione?

A linguagem humana — e os processos de pensamento envolvidos em sua geração — sempre pareceram representar uma espécie de ápice de complexidade. E, de fato, era notável que o cérebro humano — com sua rede de "meros" 100 bilhões de neurônios (e talvez 100 trilhões de conexões) — pudesse ser responsável por ela. Talvez, pudéssemos imaginar, houvesse algo mais no cérebro do que suas redes de neurônios — como alguma nova camada de física ainda não descoberta. Mas agora, com o ChatGPT, temos uma nova e importante informação: sabemos que uma rede neural artificial pura, com aproximadamente o mesmo número de conexões que o cérebro possui neurônios, é capaz de gerar linguagem humana de forma surpreendentemente eficiente.

E sim, ainda é um sistema grande e complexo — com quase tantos pesos de rede neural quanto palavras de texto disponíveis atualmente no mundo. Mas, em certo nível, ainda parece difícil acreditar que toda a riqueza da linguagem e os assuntos que ela pode abordar possam ser encapsulados em um sistema tão finito. Parte do que está acontecendo é, sem dúvida, um reflexo do fenômeno onipresente (que se tornou evidente pela primeira vez no exemplo da regra 30) de que os processos computacionais podem, na prática, amplificar consideravelmente a complexidade aparente dos sistemas, mesmo quando suas regras subjacentes são simples. Mas, na verdade, como discutimos acima, as redes neurais do tipo usado no ChatGPT tendem a ser construídas especificamente para restringir o efeito desse fenômeno — e a irredutibilidade computacional associada a ele — com o objetivo de tornar seu treinamento mais acessível.

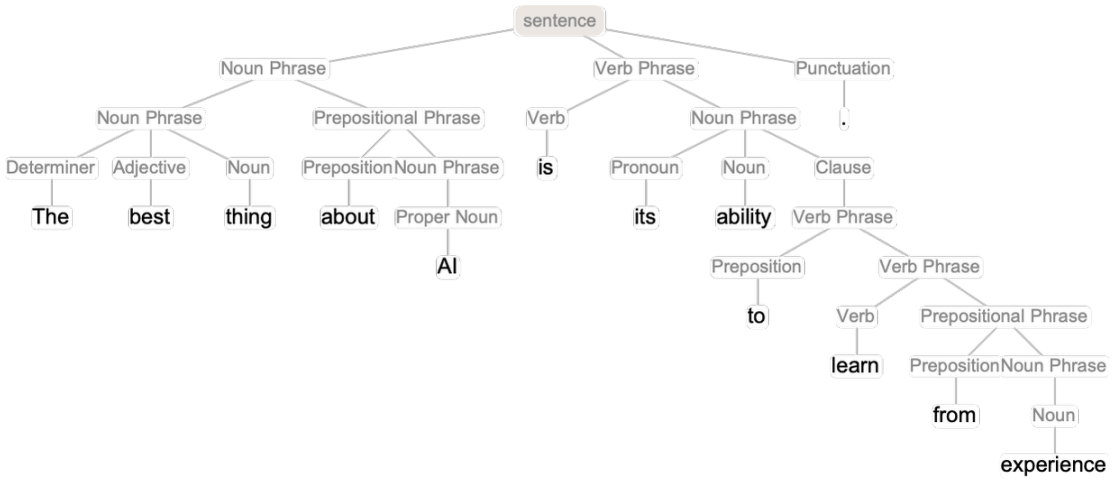
Então, como é possível que algo como o ChatGPT consiga ir tão longe com a linguagem? A resposta básica, creio eu, é que a linguagem, em um nível fundamental, é de alguma forma mais simples do que parece. E isso significa que o ChatGPT — mesmo com sua estrutura de rede neural extremamente simples — consegue "capturar a

treinamento, o ChatGPT de alguma forma "descobriu implicitamente" as regularidades na linguagem (e no pensamento) que tornam isso possível.

O sucesso do ChatGPT, creio eu, está nos fornecendo evidências de uma descoberta científica fundamental e importante: ele sugere que podemos esperar encontrar novas e importantes "leis da linguagem" — e, efetivamente, "leis do pensamento" — a serem descobertas. No ChatGPT — construído como uma rede neural — essas leis são, na melhor das hipóteses, implícitas. Mas se pudéssemos, de alguma forma, torná-las explícitas, haveria o potencial de realizar tarefas semelhantes às do ChatGPT de maneiras muito mais diretas, eficientes e transparentes.

Mas, certo, como seriam essas leis? Em última análise, elas devem nos fornecer algum tipo de receita para a forma como a linguagem — e as coisas que dizemos com ela — são construídas. Mais adiante, discutiremos como "analisar internamente o ChatGPT" pode nos dar algumas pistas sobre isso e como o que sabemos sobre a construção de linguagens computacionais sugere um caminho a seguir. Mas primeiro, vamos discutir dois exemplos já conhecidos do que equivalem a "leis da linguagem" — e como elas se relacionam com o funcionamento do ChatGPT.

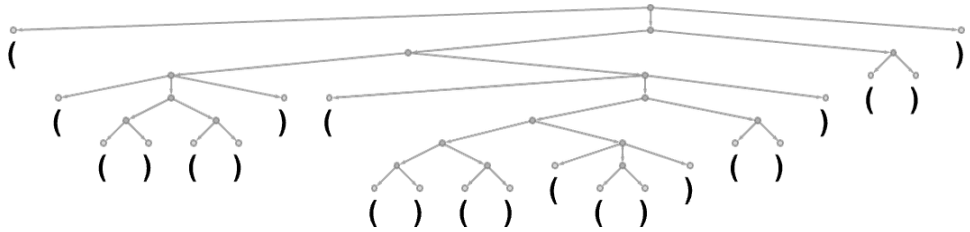
A primeira é a sintaxe da linguagem. A linguagem não é apenas uma mistura aleatória de palavras. Em vez disso, existem regras gramaticais (relativamente) definidas sobre como palavras de diferentes tipos podem ser combinadas: em inglês, por exemplo, substantivos podem ser precedidos por adjetivos e seguidos por verbos, mas normalmente dois substantivos não podem estar lado a lado. Essa estrutura gramatical pode (pelo menos aproximadamente) ser capturada por um conjunto de regras que definem como o que equivale a "árvores de análise sintática" podem ser construídas :



O ChatGPT não possui nenhum "conhecimento" explícito dessas regras. Mas, de alguma forma, durante seu treinamento, ele as "descobre" implicitamente — e então parece ser bom em segui-las. Como isso funciona? Em um nível mais amplo, não é

muito mais simples.

Considere uma “linguagem” formada por sequências de parênteses ('s e)'s, com uma gramática que especifica que os parênteses devem sempre ser balanceados, representada por uma árvore de análise sintática como:



Podemos treinar uma rede neural para produzir sequências de parênteses "gramaticalmente corretas"? Existem várias maneiras de lidar com sequências em redes neurais, mas vamos usar redes Transformer, como faz o ChatGPT. E, dada uma rede Transformer simples, podemos começar a alimentá-la com sequências de parênteses gramaticalmente corretas como exemplos de treinamento. Uma sutileza (que também aparece na geração de linguagem humana do ChatGPT) é que, além dos nossos "tokens de conteúdo" (aqui, "(" e ")"), precisamos incluir um token de "Fim", gerado para indicar que a saída não deve continuar (ou seja, para o ChatGPT, que se chegou ao "fim da história").

Se configurarmos uma rede Transformer com apenas um bloco de atenção com 8 cabeças e vetores de características de comprimento 128 (o ChatGPT também usa vetores de características de comprimento 128, mas possui 96 blocos de atenção, cada um com 96 cabeças), parece não ser possível fazê-la aprender muito sobre a linguagem entre parênteses. Mas com 2 blocos de atenção, o processo de aprendizado parece convergir — pelo menos após cerca de 10 milhões de exemplos terem sido apresentados (e, como é comum em redes Transformer, apresentar ainda mais exemplos parece apenas degradar seu desempenho).

Assim, com essa rede, podemos fazer o análogo do que o ChatGPT faz e solicitar probabilidades para qual deve ser o próximo token — em uma sequência entre parênteses:

((( ) ( ) (		
	(	46%
	)	54%
	End	0.038%

((( ) ( )))		
	(	51%
	)	15%
	End	34%

No primeiro caso, a rede está "quase certa" de que a sequência não pode terminar aqui — o que é bom, porque se terminasse, os parênteses ficariam desbalanceados. No segundo caso, porém, ela "reconhece corretamente" que a sequência pode terminar aqui, embora também "indique" que é possível "recomeçar", inserindo um "(",

pesos laboriosamente treinados, ela diz que há uma probabilidade de 15% de ter um ")" como próximo token — o que não está correto, porque isso necessariamente levaria a um parêntese desbalanceado.

Eis o que obtemos se pedirmos à rede as conclusões de maior probabilidade para sequências progressivamente mais longas de ('s:

```
( )
(( ))
((( )))
(((( ))))
((((( )))))
(((((( )))))) 
((((((( ))))))) 
(((((((( ))))))))
((((((((( )))))))))
((((((((((  ))))))))))()      (unbalanced)
((((((((((  ))))))))))        (unbalanced)
((((((((((  ))))))))))())
((((((((((  ))))))))))()))
((((((((((  ))))))))))
((((((((((  ))))))))))()))    (unbalanced)
((((((((((  ))))))))))        (unbalanced)
((((((((((  ))))))))))
((((((((((  ))))))))))
((((((((((  ))))))))))())
```

E sim, até certo ponto a rede funciona muito bem. Mas depois começa a falhar. É algo bastante típico de se ver em uma situação "precisa" como essa com uma rede neural (ou com aprendizado de máquina em geral). Casos que um humano "consegue resolver num piscar de olhos", a rede neural também consegue. Mas casos que exigem algo "mais algorítmico" (como contar explicitamente os parênteses para verificar se estão fechados) a rede neural tende a ser "computacionalmente superficial demais" para resolver de forma confiável. (Aliás, até mesmo a versão completa atual do ChatGPT tem dificuldade em identificar corretamente parênteses em sequências longas.)

Então, o que isso significa para coisas como o ChatGPT e a sintaxe de uma língua como o inglês? A linguagem dos parênteses é "austera" — e muito mais uma "história algorítmica". Mas, em inglês, é muito mais realista ser capaz de "adivinhar" o que se encaixa gramaticalmente com base em escolhas locais de palavras e outras pistas. E, sim, a rede neural é muito melhor nisso — embora talvez possa deixar passar algum caso "formalmente correto" que, bem, os humanos também podem deixar passar. Mas o ponto principal é que o fato de haver uma estrutura sintática geral na língua — com toda a regularidade que isso implica — limita, em certo sentido, "o quanto" a rede neural precisa aprender. E uma observação fundamental, quase científica, é que a arquitetura Transformer de redes neurais como a do ChatGPT parece ser capaz de aprender com sucesso o tipo de estrutura sintática em forma de árvore aninhada que parece existir (pelo menos em alguma aproximação) em todas as línguas humanas.

Uma frase como "Elétrons curiosos comem teorias azuis por peixes" está gramaticalmente correta, mas não é algo que se esperaria dizer normalmente, e não seria considerada um sucesso se o ChatGPT a gerasse — porque, bem, com os significados normais das palavras nela contidas, ela é basicamente sem sentido.

Mas existe uma maneira geral de determinar se uma frase faz sentido? Não há uma teoria geral tradicional para isso. Mas é algo que se pode considerar como tendo implicitamente "desenvolvido uma teoria" para o ChatGPT após ser treinado com bilhões de frases (presumivelmente significativas) da internet, etc.

Como seria essa teoria? Bem, há um pequeno aspecto que é conhecido há dois milênios, e esse é a lógica . E certamente na forma silogística em que Aristóteles a descobriu, a lógica é basicamente uma maneira de dizer que sentenças que seguem certos padrões são razoáveis, enquanto outras não. Assim, por exemplo, é razoável dizer "Todos os X são Y. Isto não é Y, então não é um X" (como em "Todos os peixes são azuis. Isto não é azul, então não é um peixe."). Assim como se pode imaginar, de forma um tanto fantasiosa, que Aristóteles descobriu a lógica silogística analisando (em estilo de aprendizado de máquina) inúmeros exemplos de retórica, também se pode imaginar que, no treinamento do ChatGPT, ele tenha sido capaz de "descobrir a lógica silogística" observando uma grande quantidade de textos na web, etc. (E, sim, embora se possa esperar que o ChatGPT produza textos que contenham "inferências corretas" com base em princípios como a lógica silogística, a situação é bem diferente quando se trata de lógica formal mais sofisticada — e creio que se pode esperar que ele falhe nesse caso pelos mesmos motivos que o levam a falhar na correspondência de parênteses.)

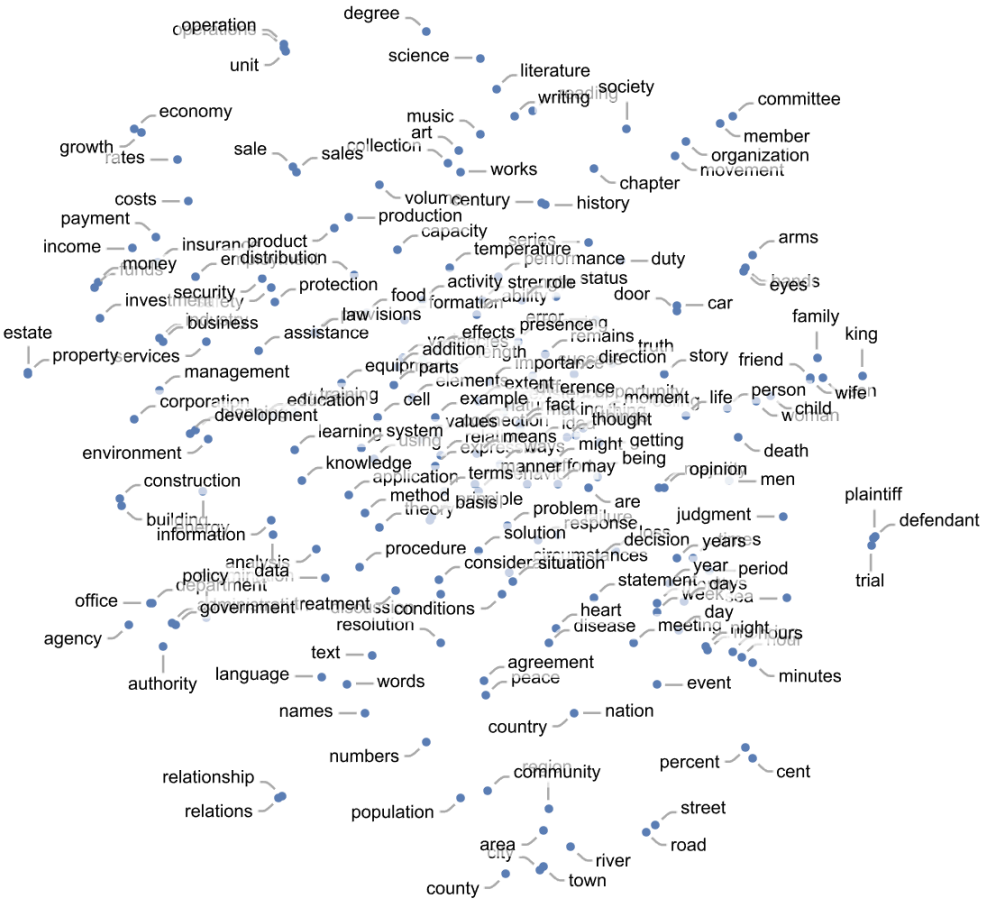
Mas, além do exemplo específico da lógica, o que se pode dizer sobre como construir (ou reconhecer) sistematicamente um texto minimamente significativo? Sim, existem coisas como o Mad Libs que usam "modelos frasais" muito específicos. Mas, de alguma forma, o ChatGPT implicitamente possui uma maneira muito mais geral de fazer isso. E talvez não haja nada a dizer sobre como isso pode ser feito além de "de alguma forma, acontece quando você tem 175 bilhões de pesos de rede neural". Mas eu suspeito fortemente que exista uma explicação muito mais simples e convincente.

Espaço de Significado e Leis Semânticas do Movimento

Discutimos acima que, dentro do ChatGPT, qualquer trecho de texto é efetivamente representado por uma matriz de números que podemos considerar como coordenadas de um ponto em algum tipo de "espaço de características linguísticas". Assim, quando o ChatGPT continua um trecho de texto, isso corresponde a traçar uma trajetória nesse espaço de características linguísticas. Mas agora podemos nos perguntar o que faz com que essa trajetória corresponda a um texto que consideramos significativo. E será que existem "leis semânticas de movimento" que definem — ou pelo menos restringem —

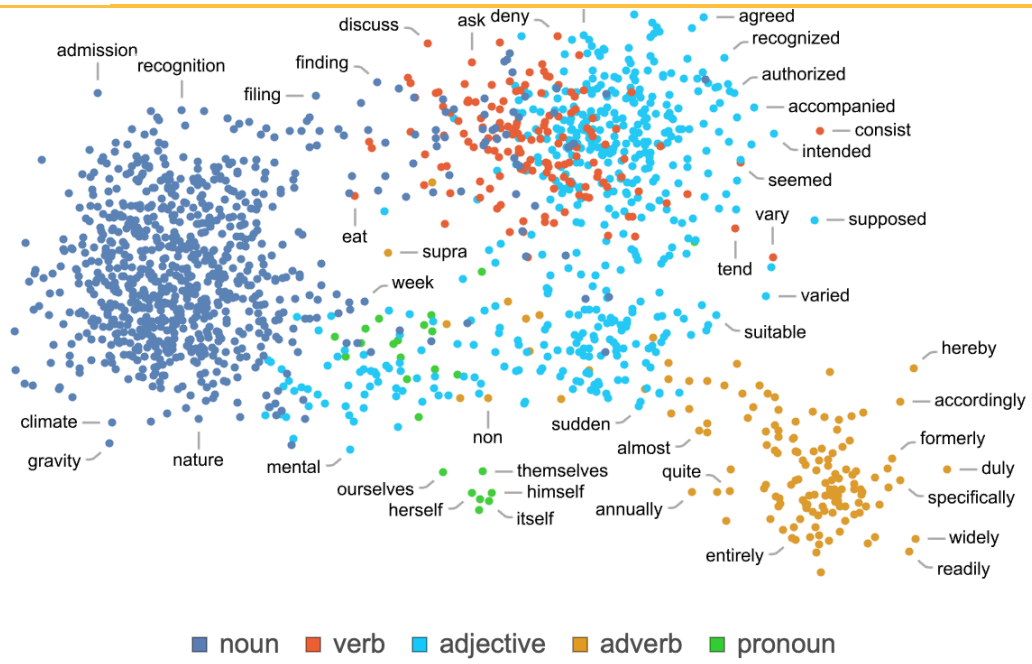
seu "significado"?

Então, como é esse espaço de características linguísticas? Aqui está um exemplo de como palavras individuais (neste caso, substantivos comuns) podem ser organizadas se projetarmos esse espaço de características em 2D:

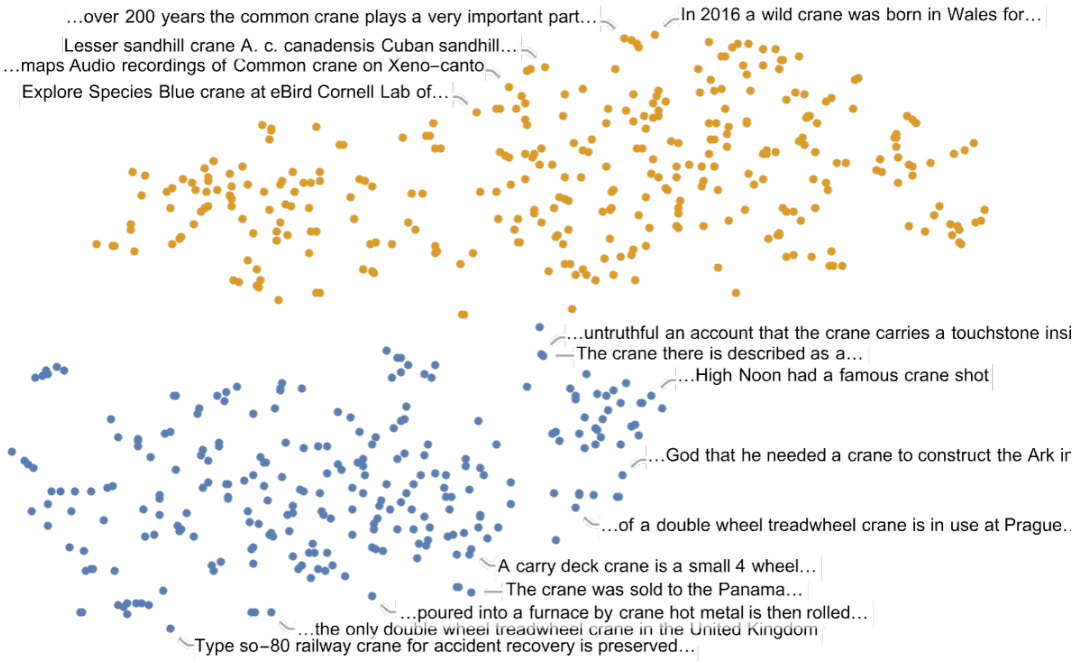


Vimos outro exemplo acima, baseado em palavras que representam plantas e animais. Mas a questão em ambos os casos é que “palavras semanticamente semelhantes” são colocadas próximas umas das outras.

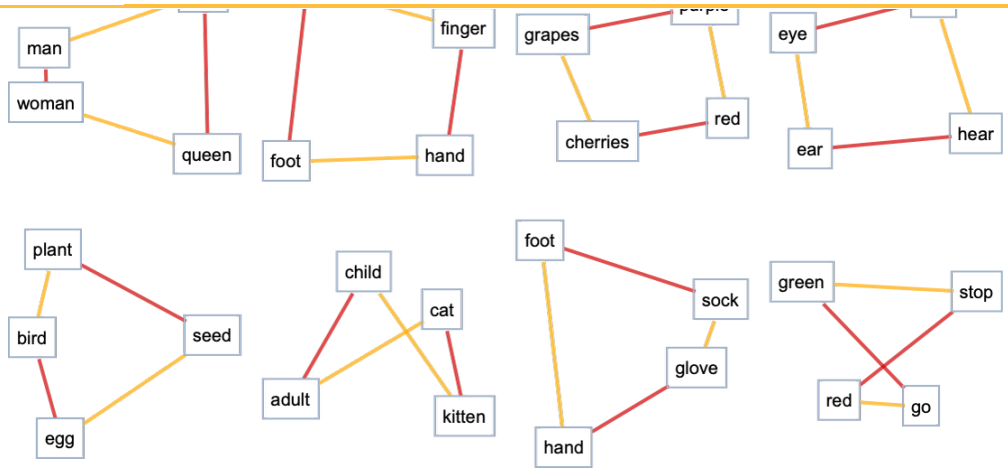
Como outro exemplo, veja como as palavras correspondentes às diferentes classes gramaticais são organizadas:



É claro que uma determinada palavra geralmente não tem apenas "um significado" (ou corresponde necessariamente a apenas uma classe gramatical). E, observando como as frases que contêm uma palavra se organizam no espaço de características, muitas vezes é possível "desvendar" diferentes significados — como no exemplo aqui para a palavra "grua" (ave ou máquina?):

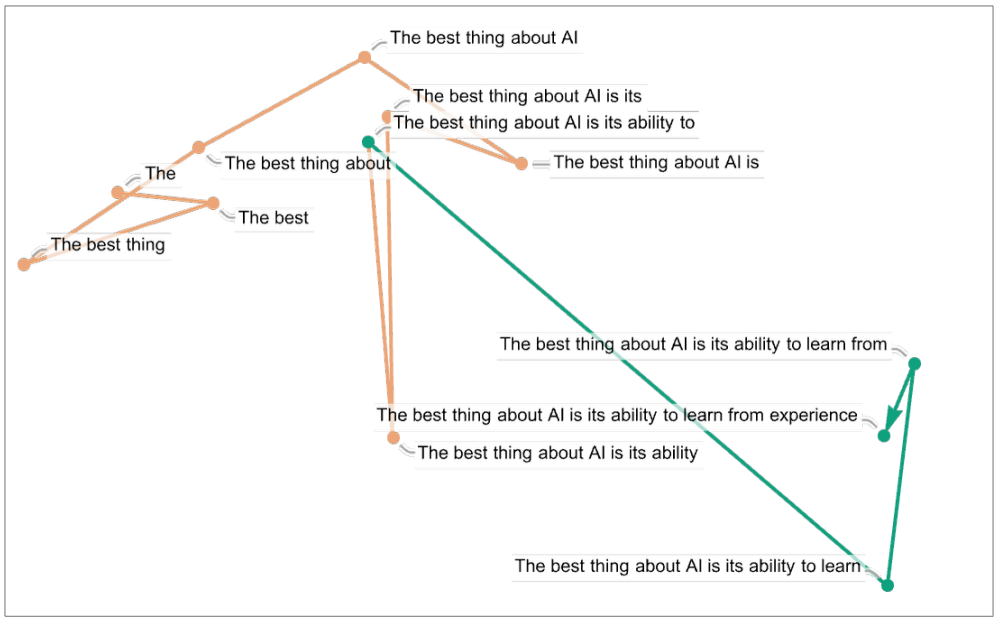


Certo, então é pelo menos plausível que possamos pensar nesse espaço de características como um espaço onde "palavras com significados semelhantes" estão localizadas próximas umas das outras. Mas que tipo de estrutura adicional podemos identificar nesse espaço? Existe, por exemplo, alguma noção de "transporte paralelo" que reflita uma "planicidade" no espaço? Uma maneira de entender isso é observar analogias:



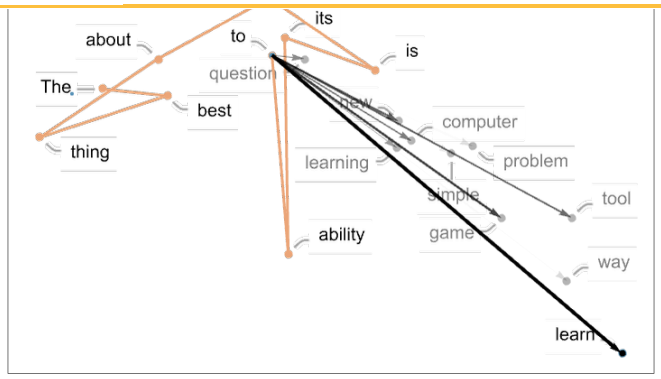
E sim, mesmo quando projetamos em 2D, muitas vezes há pelo menos um "toque de planura", embora certamente não seja algo universalmente percebido.

E quanto às trajetórias? Podemos observar a trajetória que um prompt do ChatGPT segue no espaço de características — e então podemos ver como o ChatGPT continua essa trajetória:

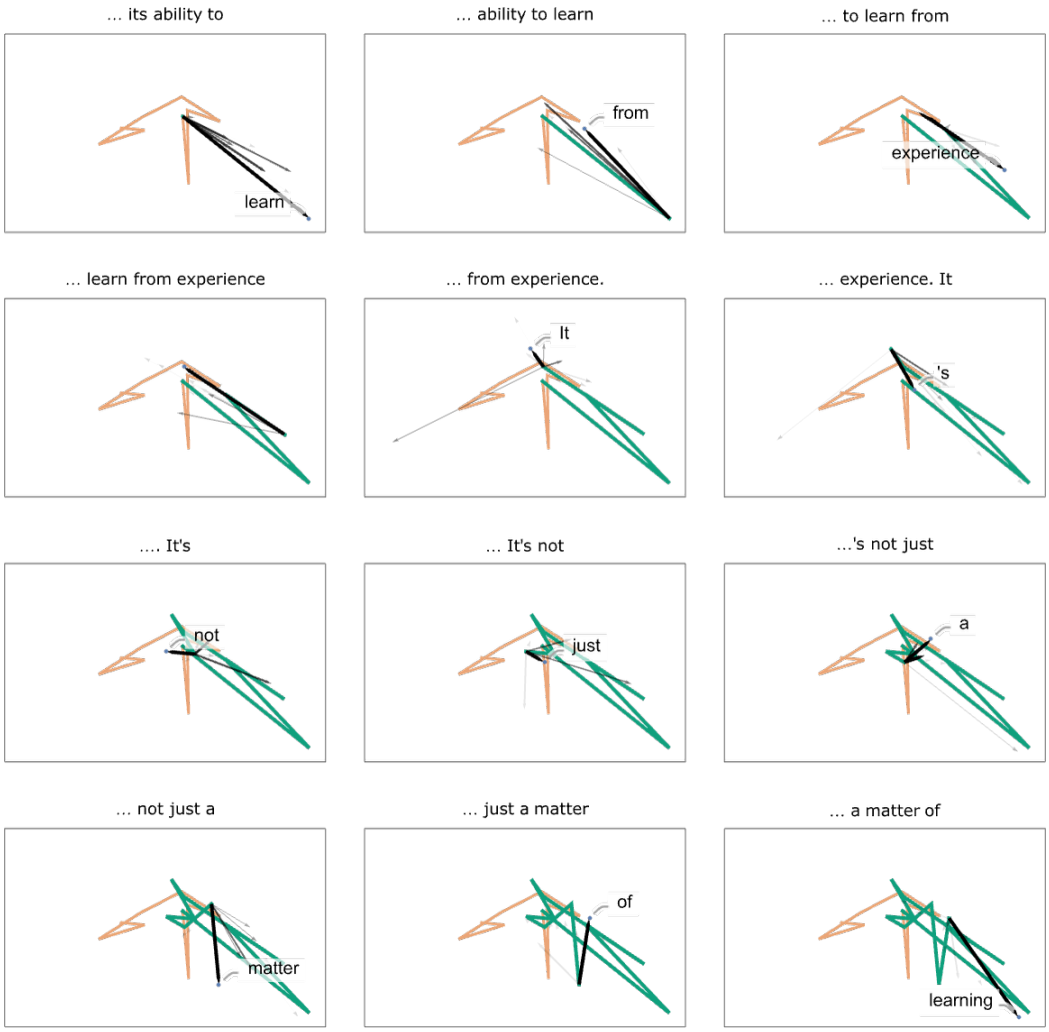


Certamente não existe aqui uma lei de movimento "geometricamente óbvia". E isso não é nada surpreendente; esperamos que seja uma história consideravelmente mais complexa . E, por exemplo, está longe de ser óbvio que, mesmo que exista uma "lei de movimento semântica" a ser encontrada, qual tipo de incorporação (ou, na verdade, quais "variáveis") ela será expressa de forma mais natural.

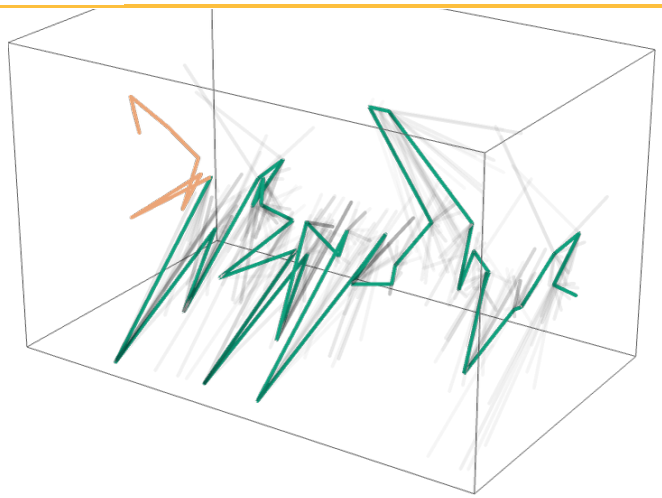
Na imagem acima, mostramos várias etapas da “trajetória” — onde, em cada etapa, escolhemos a palavra que o ChatGPT considera a mais provável (o caso de “temperatura zero”). Mas também podemos perguntar quais palavras podem “vir a seguir”, com quais probabilidades, em um determinado ponto:



E o que vemos neste caso é um "leque" de palavras de alta probabilidade que parece seguir uma direção mais ou menos definida no espaço de características. O que acontece se formos mais longe? Aqui estão os "leques" sucessivos que aparecem à medida que "avançamos" ao longo da trajetória:



Aqui está uma representação em 3D, totalizando 40 etapas:



E sim, isso parece uma bagunça — e não contribui em nada para a ideia de que se possa esperar identificar "leis semânticas do movimento" semelhantes às da física matemática estudando empiricamente "o que o ChatGPT está fazendo internamente". Mas talvez estejamos olhando para as "variáveis erradas" (ou para o sistema de coordenadas errado) e, se olhássemos para o correto, veríamos imediatamente que o ChatGPT está fazendo algo "simples como física matemática", como seguir geodésicas. Mas, por enquanto, não estamos prontos para "decodificar empiricamente", a partir de seu "comportamento interno", o que o ChatGPT "descobriu" sobre como a linguagem humana é "estruturada".

Gramática Semântica e o Poder da Linguagem Computacional

What does it take to produce "meaningful human language"? In the past, we might have assumed it could be nothing short of a human brain. But now we know it can be done quite respectably by the neural net of ChatGPT. Still, maybe that's as far as we can go, and there'll be nothing simpler—or more human understandable—that will work. But my strong suspicion is that the success of ChatGPT implicitly reveals an important "scientific" fact: that there's actually a lot more structure and simplicity to meaningful human language than we ever knew—and that in the end there may be even fairly simple rules that describe how such language can be put together.

As we mentioned above, syntactic grammar gives rules for how words corresponding to things like different parts of speech can be put together in human language. But to deal with meaning, we need to go further. And one version of how to do this is to think about not just a syntactic grammar for language, but also a semantic one.

For purposes of syntax, we identify things like nouns and verbs. But for purposes of semantics, we need "finer gradations". So, for example, we might identify the concept of "moving", and the concept of an "object" that "maintains its identity independent of location". There are endless specific examples of each of these "semantic concepts". But for the purposes of our semantic grammar, we'll just have some general kind of rule

might work (some of which I've said before). But I'll content myself here with just a few remarks that indicate some of the potential path forward.

It's worth mentioning that even if a sentence is perfectly OK according to the semantic grammar, that doesn't mean it's been realized (or even could be realized) in practice. "The elephant traveled to the Moon" would doubtless "pass" our semantic grammar, but it certainly hasn't been realized (at least yet) in our actual world—though it's absolutely fair game for a fictional world.

When we start talking about "semantic grammar" we're soon led to ask "What's underneath it?" What "model of the world" is it assuming? A syntactic grammar is really just about the construction of language from words. But a semantic grammar necessarily engages with some kind of "model of the world"—something that serves as a "skeleton" on top of which language made from actual words can be layered.

Until recent times, we might have imagined that (human) language would be the only general way to describe our "model of the world". Already a few centuries ago there started to be formalizations of specific kinds of things, based particularly on mathematics. But now there's a much more general approach to formalization: computational language.

And, yes, that's been my big project over the course of more than four decades (as now embodied in the Wolfram Language): to develop a precise symbolic representation that can talk as broadly as possible about things in the world, as well as abstract things that we care about. And so, for example, we have symbolic representations for cities and molecules and images and neural networks, and we have built-in knowledge about how to compute about those things.

And, after decades of work, we've covered a lot of areas in this way. But in the past, we haven't particularly dealt with "everyday discourse". In "I bought two pounds of apples" we can readily represent (and do nutrition and other computations on) the "two pounds of apples". But we don't (quite yet) have a symbolic representation for "I bought".

It's all connected to the idea of semantic grammar—and the goal of having a generic symbolic "construction kit" for concepts, that would give us rules for what could fit together with what, and thus for the "flow" of what we might turn into human language.

But let's say we had this "symbolic discourse language". What would we do with it? We could start off doing things like generating "locally meaningful text". But ultimately we're likely to want more "globally meaningful" results—which means "computing" more about what can actually exist or happen in the world (or perhaps in some consistent fictional world).

knowledge about lots of kinds of things. But for a complete symbolic discourse language we'd have to build in additional "calculi" about general things in the world: if an object moves from A to B and from B to C, then it's moved from A to C, etc.

Given a symbolic discourse language we might use it to make "standalone statements". But we can also use it to ask questions about the world, "Wolfram|Alpha style". Or we can use it to state things that we "want to make so", presumably with some external actuation mechanism. Or we can use it to make assertions—perhaps about the actual world, or perhaps about some specific world we're considering, fictional or otherwise.

Human language is fundamentally imprecise, not least because it isn't "tethered" to a specific computational implementation, and its meaning is basically defined just by a "social contract" between its users. But computational language, by its nature, has a certain fundamental precision—because in the end what it specifies can always be "unambiguously executed on a computer". Human language can usually get away with a certain vagueness. (When we say "planet" does it include exoplanets or not, etc.?) But in computational language we have to be precise and clear about all the distinctions we're making.

It's often convenient to leverage ordinary human language in making up names in computational language. But the meanings they have in computational language are necessarily precise—and might or might not cover some particular connotation in typical human language usage.

How should one figure out the fundamental "ontology" suitable for a general symbolic discourse language? Well, it's not easy. Which is perhaps why little has been done since the primitive beginnings Aristotle made more than two millennia ago. But it really helps that today we now know so much about how to think about the world computationally (and it doesn't hurt to have a "fundamental metaphysics" from our Physics Project and the idea of the ruliad).

But what does all this mean in the context of ChatGPT? From its training ChatGPT has effectively "pieced together" a certain (rather impressive) quantity of what amounts to semantic grammar. But its very success gives us a reason to think that it's going to be feasible to construct something more complete in computational language form. And, unlike what we've so far figured out about the innards of ChatGPT, we can expect to design the computational language so that it's readily understandable to humans.

When we talk about semantic grammar, we can draw an analogy to syllogistic logic. At first, syllogistic logic was essentially a collection of rules about statements expressed in human language. But (yes, two millennia later) when formal logic was developed, the original basic constructs of syllogistic logic could now be used to build huge "formal towers" that include, for example, the operation of modern digital circuitry. And so, we

to deal with simple patterns, expressed, say, as text. But once its whole computational language framework is built, we can expect that it will be able to be used to erect tall towers of "generalized semantic logic", that allow us to work in a precise and formal way with all sorts of things that have never been accessible to us before, except just at a "ground-floor level" through human language, with all its vagueness.

We can think of the construction of computational language—and semantic grammar—as representing a kind of ultimate compression in representing things. Because it allows us to talk about the essence of what's possible, without, for example, dealing with all the "turns of phrase" that exist in ordinary human language. And we can view the great strength of ChatGPT as being something a bit similar: because it too has in a sense "drilled through" to the point where it can "put language together in a semantically meaningful way" without concern for different possible turns of phrase.

So what would happen if we applied ChatGPT to underlying computational language? The computational language can describe what's possible. But what can still be added is a sense of "what's popular"—based for example on reading all that content on the web. But then—underneath—operating with computational language means that something like ChatGPT has immediate and fundamental access to what amount to ultimate tools for making use of potentially irreducible computations. And that makes it a system that can not only "generate reasonable text", but can expect to work out whatever can be worked out about whether that text actually makes "correct" statements about the world—or whatever it's supposed to be talking about.

So … What Is ChatGPT Doing, and Why Does It Work?

The basic concept of ChatGPT is at some level rather simple. Start from a huge sample of human-created text from the web, books, etc. Then train a neural net to generate text that's "like this". And in particular, make it able to start from a "prompt" and then continue with text that's "like what it's been trained with".

As we've seen, the actual neural net in ChatGPT is made up of very simple elements—though billions of them. And the basic operation of the neural net is also very simple, consisting essentially of passing input derived from the text it's generated so far "once through its elements" (without any loops, etc.) for every new word (or part of a word) that it generates.

But the remarkable—and unexpected—thing is that this process can produce text that's successfully "like" what's out there on the web, in books, etc. And not only is it coherent human language, it also "says things" that "follow its prompt" making use of content it's "read". It doesn't always say things that "globally make sense" (or correspond to correct computations)—because (without, for example, accessing the "computational

what things "sounded like" in its training material.

The specific engineering of ChatGPT has made it quite compelling. But ultimately (at least until it can use outside tools) ChatGPT is "merely" pulling out some "coherent thread of text" from the "statistics of conventional wisdom" that it's accumulated. But it's amazing how human-like the results are. And as I've discussed, this suggests something that's at least scientifically very important: that human language (and the patterns of thinking behind it) are somehow simpler and more "law like" in their structure than we thought. ChatGPT has implicitly discovered it. But we can potentially explicitly expose it, with semantic grammar, computational language, etc.

What ChatGPT does in generating text is very impressive—and the results are usually very much like what we humans would produce. So does this mean ChatGPT is working like a brain? Its underlying artificial-neural-net structure was ultimately modeled on an idealization of the brain. And it seems quite likely that when we humans generate language many aspects of what's going on are quite similar.

When it comes to training (AKA learning) the different "hardware" of the brain and of current computers (as well as, perhaps, some undeveloped algorithmic ideas) forces ChatGPT to use a strategy that's probably rather different (and in some ways much less efficient) than the brain. And there's something else as well: unlike even in typical algorithmic computation, ChatGPT doesn't internally "have loops" or "recompute on data". And that inevitably limits its computational capability—even with respect to current computers, but definitely with respect to the brain.

It's not clear how to "fix that" and still maintain the ability to train the system with reasonable efficiency. But to do so will presumably allow a future ChatGPT to do even more "brain-like things". Of course, there are plenty of things that brains don't do so well—particularly involving what amount to irreducible computations. And for these both brains and things like ChatGPT have to seek "outside tools"—like Wolfram Language.

But for now it's exciting to see what ChatGPT has already been able to do. At some level it's a great example of the fundamental scientific fact that large numbers of simple computational elements can do remarkable and unexpected things. But it also provides perhaps the best impetus we've had in two thousand years to understand better just what the fundamental character and principles might be of that central feature of the human condition that is human language and the processes of thinking behind it.

Thanks

I've been following the development of neural nets now for about 43 years, and during that time I've interacted with many people about them. Among them—some from long ago, some from recently,

Taliesin Beynon, Sebastian Bodenstein, Greg Brockman, Jack Cowan, Pedro Domingos, Jesse Galef, Roger Germundsson, Robert Hecht-Nielsen, Geoff Hinton, John Hopfield, Yann LeCun, Jerry Lettvin, Jerome Louradour, Marvin Minsky, Eric Mjolsness, Cayden Pierce, Tomaso Poggio, Matteo Salvarezza, Terry Sejnowski, Oliver Selfridge, Gordon Shaw, Jonas Sjöberg, Ilya Sutskever, Gerry Tesauro and Timothee Verdier. For help with this piece, I'd particularly like to thank Giulio Alessandrini and Brad Klee.

Additional Resources

"Machine Learning for Middle Schoolers" »

Introduction to Machine Learning »

Wolfram U Machine Learning Courses »

DOI: 10.31855/bc47ee6b-75c

Cite this as >

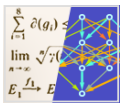
Stephen Wolfram (2023), "O que o ChatGPT está fazendo... e por que funciona?", Escritos de Stephen Wolfram. writings.stephenwolfram.com/2023/02/what-is-chatgpt-doing-and-why-does-it-work.

Publicado em: Inteligência Artificial , Linguagem e Comunicação , Novas Tecnologias

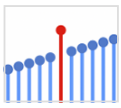
Participe da discussão

+64 comentários

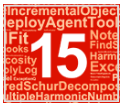
Related Writings



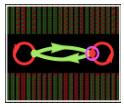
What's the Future for Pure Math Research in the Age of AI?
September 28, 2026



Towards a Theory of Bugs: The Ruliology of the Unexpected
July 21, 2026



Launching Version 15 of Wolfram Language & Mathematica: Built-in (Useful) AI & Lots of New Core Functionality
June 16, 2026



Games between Programs: The Ruliology of Competition
June 4, 2026

Escritos por Ano

2026 | 2025 | 2024 | 2023 | 2022 | 2021 | 2020 | 2019 | 2018 | 2017 | 2016 | 2015 |
2014 | 2013 | 2012 | 2011 | 2010 | 2009 | 2008 | 2007 | 2006 | 2004 | 2003 | Todos

© Stephen Wolfram, LLC | Conteúdo aberto: (código:) | Termos | RSS